

Improving Tensor Decoupling with Scaling and Smoothing

Maxime Rochkoulets

Thesis submitted for the degree of
Master of Science in Engineering: Computer Science,
option Artificial Intelligence

Supervisors

Prof. dr. ir. Hendrik Blockeel
Prof. dr. Mariya Ishteva

Assessors

Dr. Zsolt Zombori
Ir. Lorenzo Cascioli

Assistant-supervisor

Ir. Joppe De Jonghe

© 2026 KU Leuven – Faculty of Engineering Science

Published by Maxime Rochkoulets,

Department of Computer Science, Celestijnenlaan 200A bus 2402, B-3001 Leuven

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher. This publication contains the study work of a student in the context of the academic training and assessment. After this assessment no correction of the study work took place.

Preface

I would like to sincerely thank Joppe, my supervisor, whom I now consider both a mentor and a friend. I am grateful for your thoughtful feedback and guidance, for allowing me to try my own ideas (even the bad ones), and for always challenging me to understand things at a deeper level. I could not have asked for a better thesis supervisor.

Ce mémoire est dédié à mes parents, Elena et Thierry, ainsi qu'à ma soeur, Anna, pour leur amour sans faille et leur soutien inconditionnel, et pour tout ce qu'ils ont si souvent sacrifié pour moi.

Je n'aurai jamais peur d'emprunter les chemins les plus difficiles,
sachant que vous serez toujours à mes côtés.

Maxime Rochkoulets

Contents

Preface	i
Abstract	iv
List of Figures and Tables	v
1 Introduction	1
1.1 Context	1
1.2 Contributions	2
1.3 Suggested Reading Order	3
2 Tensors and Decompositions	5
2.1 Tensors	5
2.2 Operations	6
2.3 Decompositions	7
2.4 Canonical Polyadic Decomposition	8
2.5 Rank	12
2.6 Conclusion	12
3 Curve Fitting	15
3.1 Curve Fitting	15
3.2 Polynomial Interpolation	16
3.3 Splines	17
3.4 Basis Splines	18
3.5 Penalized Splines	20
3.6 Generalized Cross-Validation	21
3.7 Conclusion	22
4 Tensor Decoupling Algorithms	23
4.1 General Methodology	23
4.2 Polynomial Decoupling	25
4.3 Constrained Polynomial Decoupling	27
4.4 Coupled Matrix-Tensor Factorization	31
4.5 CMTF B-Spline Decoupling	34
4.6 Conclusion	35
5 Improved Decoupling with Scaling and Smoothing	37
5.1 Scaling	37
5.2 Smoothing	43

5.3	Algorithm (CMTF-PSD)	46
5.4	Conclusion	47
6	Application to Neural Network Compression	49
6.1	Motivation	49
6.2	Experiment	50
6.3	Results	52
6.4	Conclusion	53
7	Conclusion	55
7.1	Limitations & Future Work	55
7.2	Conclusion	57
	Bibliography	59
A	The Untangle Python Package	65
A.1	Motivation	65
A.2	Implementation Details	66
A.3	Usage	67
A.4	Future Work	68
B	Poster	69
	Code of Conduct	71

Abstract

This thesis presents two improvements to tensor decoupling algorithms, which are methods to decompose multivariate "black-box" functions into structured decoupled representations. First, a tensor scaling preprocessing step normalizes gradient magnitudes across all outputs, enabling robust decoupling of functions with highly mismatched gradient and output ranges. Second, splines coupled with a smoothing prior and generalized cross-validation are used to make the method non-parametric with respect to the number of control points. These contributions are combined into a unified new algorithm, validated on a neural network compression task, and released as Untangle, the first open-source Python library for tensor decoupling.

List of Figures and Tables

List of Figures

1.1	The tensor decoupling process aims to transform a black-box multivariate function (on the left) into a structured decoupled representation.	1
1.2	The decoupling viewed as a mapping to a multilayer perceptron.	2
1.3	Suggested reading order.	3
2.1	Real vector, matrix, and 3-dimensional tensor.	5
2.2	The mode-1 unfolding of a 3-dimensional tensor.	6
2.3	The 3-way CP decomposition viewed as a sum of vector outer-products. . . .	8
3.1	Runge’s phenomenon in polynomial interpolation.	17
3.2	B-splines of order 2 and 3 applied to Runge’s function.	19
3.3	Importance of choosing the right number of control points with B-splines. . .	20
3.4	GCV automatically finds the λ value that best balances error and smoothness.	21
4.1	The tensor decoupling process aims to transform a black-box multivariate function (on the left) into a structured decoupled representation.	23
4.2	When a constraint is not enforced on factor $\mathbf{H}$ during the CPD, its columns are noisy and discontinuous, and hardly represent the derivatives of our internals.	28
4.3	When a constraint is enforced during the CPD, the columns of $\mathbf{H}$ effectively represent the derivatives of our internals, and can be interpolated.	31
5.1	Per-output error as a function of df . As the number of basis functions increases, B-splines tend to overfit while the P-spline projection stays stable and consistent.	45
6.1	Tensor decoupling can be seen as mapping a black-box multivariate function to a one hidden layer neural network with flexible activations and no bias terms.	49
6.2	Neural network compression with tensor decoupling.	51
6.3	Internals (activation functions) learned by the rank-32 decoupling.	53
A.1	The logo of the Untangle package.	65

A.2	Enforcing tensor shapes at runtime in the signature of our CP-ALS function. . .	66
A.3	An example of function annotated with the <code>@jax.jit</code> decorator.	67
A.4	Example of using the Untangle library to decouple a cubic polynomial.	68

List of Tables

5.1	Results of applying CMTF-BSD to $\mathbf{f}^{(1)}$ (averaged over 20 random initializations).	38
5.2	Results of applying CMTF-BSD to $\mathbf{f}^{(2)}$ (averaged over 20 random initializations).	38
5.3	Results of decoupling $\mathbf{f}^{(1)}$ and $\mathbf{f}^{(2)}$, comparing the two scaling approaches. Metrics averaged over 20 random initializations.	42
6.1	Neural network compression results for different decoupling ranks.	52

Chapter 1

Introduction

1.1 Context

In this work we are interested in the decomposition of multivariate, vector-valued functions $\mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^n$. By collecting information about a function and applying a *tensor decoupling algorithm*, we seek to obtain a representation of the form

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{W} \mathbf{g}(\mathbf{V}^T \mathbf{x}), \quad (1.1)$$

where $\mathbf{W} \in \mathbb{R}^{n \times r}$, $\mathbf{V} \in \mathbb{R}^{m \times r}$ and $\mathbf{g} : \mathbb{R}^r \rightarrow \mathbb{R}^r$, with r the *rank* of the decomposition.

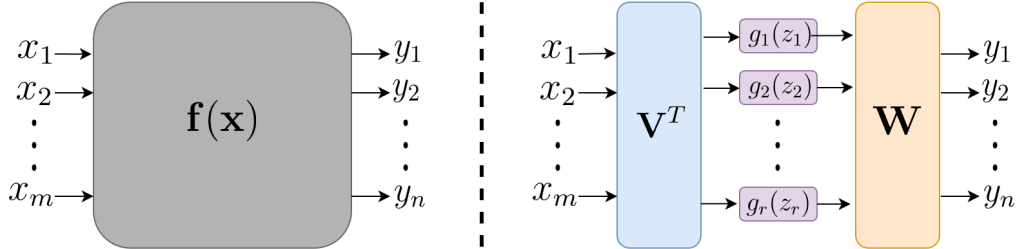


FIGURE 1.1: The tensor decoupling process aims to transform a black-box multivariate function (on the left) into a structured decoupled representation.

We observe that equation (1.1) represents $\mathbf{f}(\mathbf{x})$ as a linear mapping, followed by an element-wise (typically non-linear) transformation $\mathbf{g}$, followed by another linear mapping. In this context, $\mathbf{g}$ is referred to as the *collection of internal univariates*. We note that the rank of the decomposition defines the number of such internals. This is illustrated in Figure 1.1.

It is also easy to see that equation (1.1) defines a two-layer multilayer perceptron (MLP), with m inputs, r hidden neurons and n outputs. Thus, this procedure can be seen as the transformation of $\mathbf{f}$ into a neural-network-like representation (Figure 1.2). One important difference is that in tensor decoupling, activations are flexible and learned from data, meaning that typically $g_i(x) \neq g_j(x)$ for $i \neq j$, while nonlinearities in neural networks are usually identical and parallelizable.

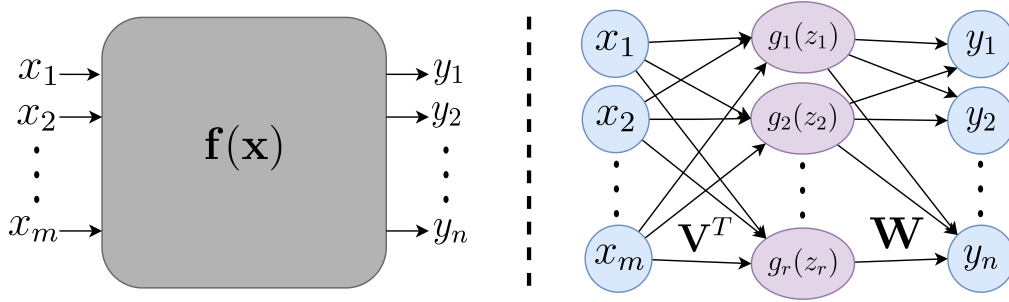


FIGURE 1.2: The decoupling viewed as a mapping to a multilayer perceptron.

Tensor decoupling was first introduced by Dreesen et al. [2015], focusing on the polynomial case. It uses a tensor canonical polyadic decomposition applied to a 3-dimensional tensor of stacked Jacobian matrices. Since then, many approaches have been proposed to improve this algorithm, focusing on making it more robust in noisy settings [Hollander, 2017, Decuyper et al., 2021], using additional information [Zniyed et al., 2021] or different internal representations [De Jonghe and Ishteva, 2025]. A selection of these algorithms is described in detail in Chapter 4.

Finally, tensor decoupling methods have many practical applications. While they were originally used in signal processing and nonlinear system identification, they have recently gained interest in the field of deep learning. A practical example of using these methods for neural network compression is presented in Chapter 6.

1.2 Contributions

This work presents three main contributions.

First and foremost, we propose a simple and computationally cheap improvement to all existing tensor decoupling methods via a rescaling of the input Jacobian tensor. We also discuss and compare this approach to a direct scaling of the target function. This preprocessing step is particularly useful for functions with varying gradient magnitudes, allowing us to work *out-of-the-box* with any multivariate function.

We then show that using penalized splines to model our internals, coupled with generalized cross-validation, allows us to increase their degrees of freedom up to the number of samples. This makes our method non-parametric with respect to this hyperparameter, and maximizes the expressivity of our internals without overfitting, which is difficult to control with standard B-spline based solutions.

The final contribution is the release of an open-source Python package implementing the discussed algorithms, called Untangle¹. Published under MIT license and built on top of Jax [Bradbury et al., 2018], it aims to democratize tensor decoupling methods in the era of deep learning. This library is easy to use and fast, it is also the first tensor decoupling

¹<https://github.com/mrochk/untangle>

library ever written in Python. It served as the primary tool to perform the experiments discussed in this thesis. Selected implementation details are discussed in Appendix A.

1.3 Suggested Reading Order

Figure 1.3 shows a suggested order in which this thesis may be explored. Chapters displayed on the same level can be read concurrently as they do not depend on each other.

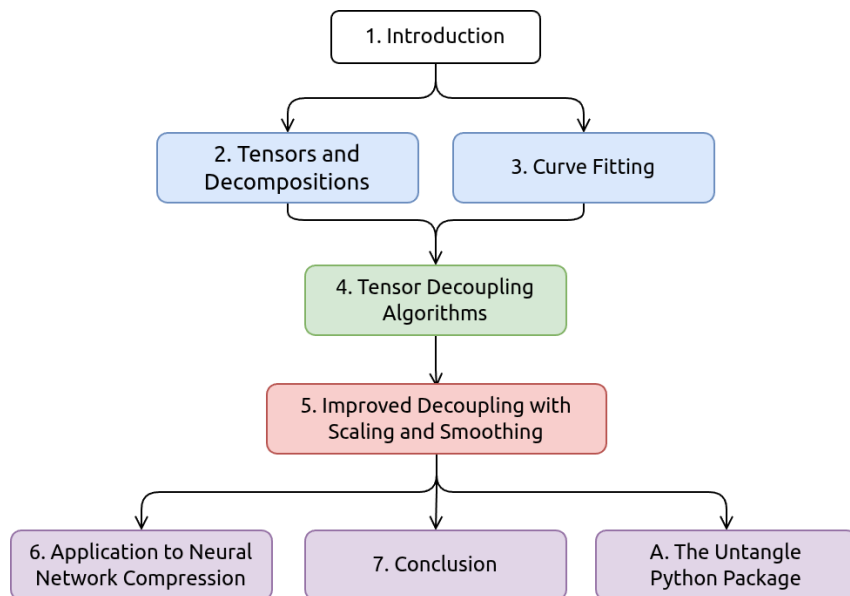


FIGURE 1.3: Suggested reading order.

Chapter 2

Tensors and Decompositions

In this chapter, we introduce tensors as a natural extension of vectors and matrices to higher-order data, and discuss properties of tensors that are relevant in the context of this work. We then take a look at tensor decompositions, focusing on the canonical polyadic decomposition and how it can be computed in practice.

2.1 Tensors

Tensors are generalizations of vectors and matrices to higher dimensions: they are *multidimensional arrays*. The number of indices of a tensor is referred to as its *order* or *dimension*. In that sense, scalars, vectors and matrices are tensors of order 0, 1, and 2, respectively. To refer to a specific dimension of a tensor, we use the term *mode*. Tensors are typically denoted with uppercase Euler letters, such as $\mathcal{X}$, $\mathcal{T}$, or $\mathcal{J}$.

Although in this work we will be dealing with tensors of order at most 3, a tensor generalizes to any number of dimensions. A tensor of order k containing real entries is defined as $\mathcal{T} \in \mathbb{R}^{d_1 \times \dots \times d_k}$. It follows that the total number of elements of a tensor is the product of the size of its dimensions $|\mathcal{T}| = d_1 \times \dots \times d_k$.

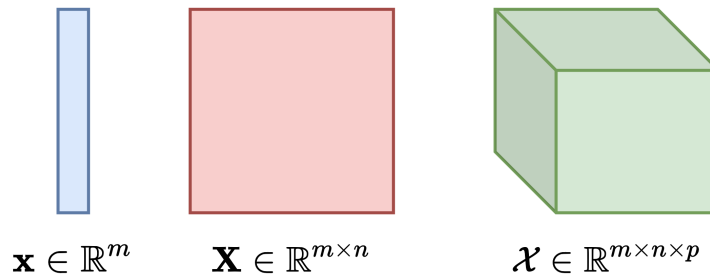


FIGURE 2.1: Real vector, matrix, and 3-dimensional tensor.

Higher-order tensors (of order ≥ 3) arise often in real-life scenarios, such as when dealing with data that is naturally organized in more than two dimensions. This could be a set of

measurements arranged in $\{\text{object} \times \text{feature} \times \text{scenario}\}$. This makes tensors well suited for data science applications, where they often arise naturally.

For us, tensors are useful for another reason, as we will show in Chapter 4 that we can find the decoupled representation of a function [equation (1.1)] by computing the decomposition of a tensor of stacked Jacobian matrices, evaluated in a number of operating points.

2.2 Operations

This section introduces some relevant operations used throughout this work.

2.2.1 Khatri-Rao Product

Given two matrices $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{n \times r}$ with the same number of columns, their *Khatri-Rao product* $(\mathbf{A} \odot \mathbf{B}) \in \mathbb{R}^{mn \times r}$ is defined as

$$\mathbf{A} \odot \mathbf{B} = [\mathbf{a}_1 \otimes \mathbf{b}_1 \quad \mathbf{a}_2 \otimes \mathbf{b}_2 \quad \cdots \quad \mathbf{a}_r \otimes \mathbf{b}_r], \quad (2.1)$$

where $\mathbf{a}_i$ and $\mathbf{b}_i$ denote the i -th columns of $\mathbf{A}$ and $\mathbf{B}$, respectively, and $\otimes$ denotes the Kronecker product. Explicitly, each column $\mathbf{a}_i \otimes \mathbf{b}_i = (\mathbf{A} \odot \mathbf{B})_i$ is defined as

$$\mathbf{a}_i \otimes \mathbf{b}_i = \begin{bmatrix} a_{1i} \mathbf{b}_i \\ a_{2i} \mathbf{b}_i \\ \vdots \\ a_{mi} \mathbf{b}_i \end{bmatrix} = \begin{bmatrix} a_{1i} b_{1i} \\ a_{1i} b_{2i} \\ \vdots \\ a_{1i} b_{ni} \\ a_{2i} b_{1i} \\ \vdots \\ a_{mi} b_{ni} \end{bmatrix}. \quad (2.2)$$

2.2.2 Unfolding

Tensor *unfolding*, also called *matricization*, is the reshaping of the tensor into a matrix.

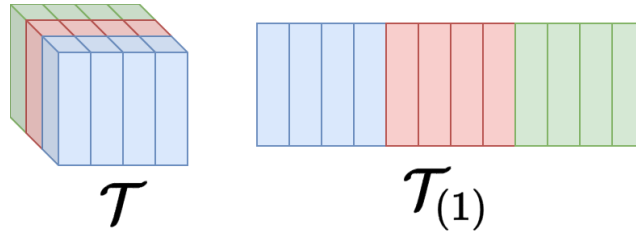


FIGURE 2.2: The mode-1 unfolding of a 3-dimensional tensor.

Unfolding, illustrated for the $k = 1$ case in Figure 2.2, is defined with respect to a mode of the tensor, and the mode- k unfolding of a tensor $\mathcal{T}$ is commonly written as $\mathcal{T}_{(k)}$. For

3-dimensional tensors, the three possible unfoldings are defined as follows

$$\begin{aligned}
\mathcal{T}_{(1)} &= \begin{bmatrix} t_{111} & t_{121} & \dots & t_{1np} \\ t_{211} & t_{221} & \dots & t_{2np} \\ \vdots & \vdots & \ddots & \vdots \\ t_{m11} & t_{m21} & \dots & t_{mnp} \end{bmatrix} & \mathcal{T}_{(1)} \in \mathbb{R}^{m \times np}, \\
\mathcal{T}_{(2)} &= \begin{bmatrix} t_{111} & t_{211} & \dots & t_{m1p} \\ t_{121} & t_{221} & \dots & t_{m2p} \\ \vdots & \vdots & \ddots & \vdots \\ t_{1n1} & t_{2n1} & \dots & t_{mnp} \end{bmatrix} & \mathcal{T}_{(2)} \in \mathbb{R}^{n \times mp}, \\
\mathcal{T}_{(3)} &= \begin{bmatrix} t_{111} & t_{211} & \dots & t_{mn1} \\ t_{112} & t_{211} & \dots & t_{mn2} \\ \vdots & \vdots & \ddots & \vdots \\ t_{11p} & t_{21p} & \dots & t_{mnp} \end{bmatrix} & \mathcal{T}_{(3)} \in \mathbb{R}^{p \times mn}.
\end{aligned} \tag{2.3}$$

2.2.3 Norm

The norm of a tensor is analogous to the matrix Frobenius norm since it can be written as the L2-norm of the vectorized tensor

$$\|\mathcal{T}\| = \|\text{vec}(\mathcal{T})\|_2 = \sqrt{\sum_{i=1}^m \sum_{j=1}^n \sum_{k=1}^p t_{ijk}^2}. \tag{2.4}$$

2.3 Decompositions

As their name suggests, *tensor decompositions* [Kolda and Bader, 2009, Ballard and Kolda, 2025] are similar to matrix decompositions, since they seek a representation of the tensor as a set of one or more other tensors. These tensors can then be combined to (often only approximately) reconstruct the original tensor. In fact, a lot of tensor decompositions can be seen as generalizations of matrix decompositions.

The goal of computing a decomposition can be to find a smaller, more memory-efficient representation of our tensor [Cichocki et al., 2015], as the number of elements of a tensor grows exponentially with its number of dimensions. If a good approximation can be computed, it might be more beneficial to store this smaller representation rather than the original tensor.

By *good approximation*, we generally mean a decomposition $\hat{\mathcal{T}}$ that has a low error ϵ with respect to our *target* $\mathcal{T}$, which is measured as follows

$$\epsilon = \frac{\|\mathcal{T} - \hat{\mathcal{T}}\|}{\|\mathcal{T}\|}, \tag{2.5}$$

where dividing by the norm [equation (2.4)] gives us a relative error, which is not conditioned on the magnitude of the entries in the target.

Another benefit of computing a decomposition is in the context of data analysis and interpretation [Ballard and Kolda, 2025]. Indeed, just as we can learn more about a matrix by computing a principal component analysis, components returned by a tensor decomposition can be visualized and may provide insights about underlying patterns in our dataset.

2.4 Canonical Polyadic Decomposition

The canonical polyadic decomposition is a fundamental building block for the rest of this work, we now take the time to describe it in detail.

2.4.1 Definition

The Canonical Polyadic Decomposition [Harshman et al., 1970, Carroll and Chang, 1970] seeks to approximate a higher-order tensor as a sum of rank-1 components. We again focus on the 3-way case, although this decomposition works with tensors of any order.

The CP decomposition seeks to express a target tensor $\mathcal{T} \in \mathbb{R}^{m \times n \times p}$ in terms of three *factors*, which are matrices $\mathbf{A} \in \mathbb{R}^{m \times r}$, $\mathbf{B} \in \mathbb{R}^{n \times r}$ and $\mathbf{C} \in \mathbb{R}^{p \times r}$. The outer-product of the vectors making up the columns of these matrices form the rank-1 *components*. The scalar $r \in \mathbb{N}$ is the CPD *rank* and defines the size of the second dimension of the factors.

The CP decomposition of a tensor $\mathcal{T}$ is commonly written as follows

$$\mathcal{T} \approx \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket, \quad (2.6)$$

and can be expressed as a sum of r rank-1 tensors

$$\llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket = \sum_{i=1}^r \mathbf{a}_i \circ \mathbf{b}_i \circ \mathbf{c}_i, \quad (2.7)$$

where $\circ$ denotes the vector outer-product. This is illustrated in Figure 2.3.

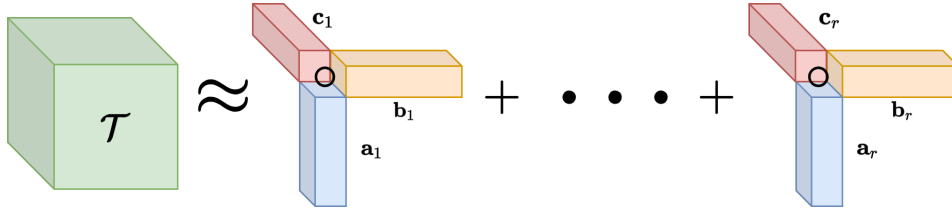


FIGURE 2.3: The 3-way CP decomposition viewed as a sum of vector outer-products.

The CPD is also sometimes computed with explicit scalar weights λ_j for each component [Kolda and Bader, 2009], which can be useful for normalizing the component vectors. In that case, the CP decomposition is defined as

$$\llbracket \boldsymbol{\lambda}; \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket = \sum_{i=1}^r \lambda_i (\mathbf{a}_i \circ \mathbf{b}_i \circ \mathbf{c}_i), \quad (2.8)$$

with $\|\mathbf{a}_i\| = \|\mathbf{b}_i\| = \|\mathbf{c}_i\| = 1$ for $i = 1 \dots r$.

2.4.2 Uniqueness

Looking at equation (2.7), it is straightforward to see that the columns of the factors can be arbitrarily permuted and still yield the same result, since the order in which the rank-1 components are summed does not matter. Furthermore, if we rewrite equation (2.7) as

$$\mathcal{T} \approx \sum_{i=1}^r (a_i \mathbf{a}_i) \circ (b_i \mathbf{b}_i) \circ (c_i \mathbf{c}_i), \quad (2.9)$$

with $a_i b_i c_i = 1$, we see that the components can be arbitrarily rescaled. What we observe here is that the CP decomposition is unique only up to arbitrary scaling and permutation of the factors, which is called *essential uniqueness*.

The conditions under which the CPD is guaranteed to be essentially unique have been derived by Kruskal [1977]. If we denote by $k(\mathbf{A})$ the *Kruskal rank* of a matrix $\mathbf{A}$, which is defined to be the largest k for which any set of k columns of $\mathbf{A}$ are linearly independent, we have that the rank- r CP decomposition $\llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket$ is essentially unique under

$$k(\mathbf{A}) + k(\mathbf{B}) + k(\mathbf{C}) \geq 2r + 2. \quad (2.10)$$

From equation (2.10) it immediately follows that a simple sufficient condition is that the factors are full column-rank. This contrasts with the matrix case, where a rank- r factorization is never unique.

2.4.3 Computing the CP Decomposition

In general, when we compute the CP decomposition of a target tensor $\mathcal{T}$, our goal is to minimize the error as we defined it in equation (2.5)

$$\epsilon = \frac{\|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|}{\|\mathcal{T}\|}, \quad (2.11)$$

thus, we may frame our optimization problem as follows

$$\min_{(\mathbf{A}, \mathbf{B}, \mathbf{C})} \frac{\|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|}{\|\mathcal{T}\|}, \quad (2.12)$$

which we typically further rewrite as

$$\begin{aligned} & \min_{(\mathbf{A}, \mathbf{B}, \mathbf{C})} \|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|^2, \\ & \text{subject to } \mathbf{A} \in \mathbb{R}^{m \times r}, \mathbf{B} \in \mathbb{R}^{n \times r}, \mathbf{C} \in \mathbb{R}^{p \times r}, \end{aligned} \quad (2.13)$$

where we dropped the denominator and squared the objective, as neither affects the minimizer. The goal of all CPD algorithms is to solve equation (2.13), with some optional constraints.

There are different approaches to tackle this problem, but in this work, we will focus on the CP-ALS method [Harshman et al., 1970, Carroll and Chang, 1970], where ALS stands for *alternating least-squares*. It is arguably the most widely used approach for computing the CP decomposition [Kolda and Bader, 2009].

The procedure can be basically described as follows:

- (0) Initialize the factors $(\mathbf{A}, \mathbf{B}, \mathbf{C})$.
- (1) Find $\mathbf{A}$ that minimizes equation (2.13) with $\mathbf{B}$ and $\mathbf{C}$ fixed.
- (2) Find $\mathbf{B}$ that minimizes equation (2.13) with $\mathbf{A}$ and $\mathbf{C}$ fixed.
- (3) Find $\mathbf{C}$ that minimizes equation (2.13) with $\mathbf{A}$ and $\mathbf{B}$ fixed.
- (4) If not converged, go back to step (1), otherwise return $(\mathbf{A}, \mathbf{B}, \mathbf{C})$.

This algorithm has the property that the error is always monotonically decreasing, since we modify a factor only if it leads to a lower global error ϵ . In practice, we measure convergence by comparing the errors ϵ_t to ϵ_{t-1} , and stop if the decrease is lower than a certain threshold.

To be able to solve for step (1), (2) and (3), we can use a property of *Kruskal tensors*, which are tensors that can be expressed as a sum of rank-1 tensors (we implicitly assume that $\mathcal{T}$ is a Kruskal tensor when we compute its CPD). This property [Kolda and Bader, 2009] states that the mode- k unfoldings [equation (2.3)] of a Kruskal tensor $\mathcal{K} = \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket$ can be written as

$$\begin{aligned}\mathcal{K}_{(1)} &= \mathbf{A}(\mathbf{C} \odot \mathbf{B})^T, \\ \mathcal{K}_{(2)} &= \mathbf{B}(\mathbf{C} \odot \mathbf{A})^T, \\ \mathcal{K}_{(3)} &= \mathbf{C}(\mathbf{B} \odot \mathbf{A})^T,\end{aligned}\tag{2.14}$$

that is, the mode- k unfolding of a (3-dimensional) Kruskal tensor $\mathcal{K}$ can be written as the product of the k -th factor times the Khatri-Rao product [equation (2.1)] of the two others.

If we plug in our CPD factors, we have that

$$\begin{aligned}\min_{\mathbf{A}} \|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|^2 &\equiv \min_{\mathbf{A}} \|\mathcal{T}_{(1)} - \mathbf{A}(\mathbf{C} \odot \mathbf{B})^T\|_F^2, \\ \min_{\mathbf{B}} \|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|^2 &\equiv \min_{\mathbf{B}} \|\mathcal{T}_{(2)} - \mathbf{B}(\mathbf{C} \odot \mathbf{A})^T\|_F^2, \\ \min_{\mathbf{C}} \|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|^2 &\equiv \min_{\mathbf{C}} \|\mathcal{T}_{(3)} - \mathbf{C}(\mathbf{B} \odot \mathbf{A})^T\|_F^2.\end{aligned}\tag{2.15}$$

Using the formulation given by equation (2.15) is useful because it shows that solving for each factor is a linear least-squares problem, for which we can derive a closed-form solution (here for factor $\mathbf{A}$)

$$\begin{aligned}0 &= \frac{\partial}{\partial \mathbf{A}} \|\mathcal{T}_{(1)} - \mathbf{A}(\mathbf{C} \odot \mathbf{B})^T\|_F^2, \\ 0 &= -2\mathcal{T}_{(1)}(\mathbf{C} \odot \mathbf{B}) + 2\mathbf{A}(\mathbf{C} \odot \mathbf{B})^T(\mathbf{C} \odot \mathbf{B}), \\ \mathbf{A} &= \mathcal{T}_{(1)}(\mathbf{C} \odot \mathbf{B})[(\mathbf{C} \odot \mathbf{B})^T(\mathbf{C} \odot \mathbf{B})]^{-1}.\end{aligned}\tag{2.16}$$

Finally, using a property of the Khatri-Rao product and Gram matrices, we can find an equivalent expression that is more efficient to compute

$$\mathbf{A} = \mathcal{T}_{(1)}(\mathbf{C} \odot \mathbf{B})(\mathbf{B}^T \mathbf{B} * \mathbf{C}^T \mathbf{C})^{-1},\tag{2.17}$$

where $*$ is the Hadamard (elementwise) product, giving us the final expression for the core operation of CP-ALS, which we turn into a reusable function in Algorithm 1.

Algorithm 1 Function for solving a single CP-ALS step.

```

function FACTOR-SOLVE( $\mathcal{T}_{(k)} \in \mathbb{R}^{m \times np}$ ,  $\mathbf{X} \in \mathbb{R}^{n \times r}$ ,  $\mathbf{Y} \in \mathbb{R}^{p \times r}$ )
     $\mathbf{K} \leftarrow (\mathbf{Y} \odot \mathbf{X})$  ▷ Khatri-Rao product.
     $\mathbf{G} \leftarrow (\mathbf{X}^T \mathbf{X}) * (\mathbf{Y}^T \mathbf{Y})$  ▷ Hadamard product of Gram matrices.
    return  $\mathcal{T}_{(k)} \mathbf{K} \mathbf{G}^{-1}$  ▷ Solution that minimizes equation (2.13).
end function
    
```

We may now derive the full procedure in Algorithm 2. We initialize our factors by sampling at random from a standard normal distribution, which leads (almost surely) to full column-rank factors that satisfy equation (2.10) at the start, although other initializations are possible [Ballard and Kolda, 2025].

Algorithm 2 Full CP-ALS decomposition procedure.

```

function CP-ALS( $\mathcal{T} \in \mathbb{R}^{m \times n \times p}$ ,  $r \in \mathbb{N}$ ,  $T \in \mathbb{N}$ ,  $\tau \in \mathbb{R}$ )
     $(\mathbf{A}, \mathbf{B}, \mathbf{C}) \sim \mathcal{N}(0, 1)$  ▷ Step (0)
    for  $t = 1 \dots T$  do
         $\mathbf{A} \leftarrow \text{Factor-Solve}(\mathcal{T}_{(1)}, \mathbf{B}, \mathbf{C})$  ▷ Step (1) – Algorithm 1
         $\mathbf{B} \leftarrow \text{Factor-Solve}(\mathcal{T}_{(2)}, \mathbf{A}, \mathbf{C})$  ▷ Step (2) – Algorithm 1
         $\mathbf{C} \leftarrow \text{Factor-Solve}(\mathcal{T}_{(3)}, \mathbf{A}, \mathbf{B})$  ▷ Step (3) – Algorithm 1
         $\epsilon_t \leftarrow \frac{1}{\|\mathcal{T}\|} \|\mathcal{T} - \llbracket \mathbf{A}, \mathbf{B}, \mathbf{C} \rrbracket\|$  ▷ Equation (2.5)
        if  $(\epsilon_{t-1} - \epsilon_t) < \tau$  then
            break ▷ Step (4)
        end if
    end for
    return  $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ 
end function
    
```

The CP optimization problem [equation (2.13)] is nonconvex and therefore CP-ALS is not guaranteed to converge to a global minimizer. To mitigate this problem, we usually run the procedure multiple times and return the decomposition with the lowest error.

2.4.4 Practical Applications

The canonical polyadic decomposition is one of the most popular and widely used tensor decompositions. It can be used for compression, but its most interesting use cases are in data analysis [Ballard and Kolda, 2025], where it can be used to extract patterns from high-dimensional datasets without first requiring the data to be reduced into a matrix, which typically leads to information loss.

More precisely, the components of the CPD can be read as co-occurring patterns across all modes. If explicit weights are returned [equation (2.8)], they can be used to rank components by importance, similarly to singular values in a principal component analysis.

2.5 Rank

Although tensors are the natural extension of matrices to higher dimensions, there are several notions that do not translate directly (and intuitively) from matrices. The concept of *rank* is one of them. We conclude this chapter by discussing the general concept of tensor rank, and linking tensor rank to CPD rank.

2.5.1 Matrix Rank

The rank of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, denoted as $\text{rank}(\mathbf{A})$, corresponds to the dimension of the subspace spanned by its columns (or rows). In other words, the number of linearly independent columns of $\mathbf{A}$. Another useful definition of matrix rank is that it is the smallest number of vector outer products that can be summed to produce the matrix

$$\mathbf{A} = \sum_{i=1}^r \mathbf{v}_i \mathbf{w}_i^T, \quad (2.18)$$

with $r = \text{rank}(\mathbf{A})$. From equation (2.18) it follows that

$$\text{rank}(\mathbf{A}) \leq \min(m, n). \quad (2.19)$$

2.5.2 Tensor Rank

However, and perhaps counter-intuitively, the tensor rank does not satisfy equation (2.19). In fact, it is entirely possible that

$$\text{rank}(\mathcal{T}) \gg \max(d_1, \dots, d_k), \quad (2.20)$$

for a tensor $\mathcal{T}$ of order $k \geq 3$.

Additionally, finding the rank of a tensor is NP-hard [Håstad, 1989, Hillar and Lim, 2013], contrary to the matrix rank, which can be easily computed via its singular value decomposition. Furthermore, the best rank- r approximation of a matrix can be directly found with a truncated SVD [Eckart and Young, 1936], while for a tensor, a best low-rank approximation is not even guaranteed to exist [De Silva and Lim, 2008].

The notion of tensor rank as presented here is closely related to the CP decomposition presented earlier. In fact, we have that $\text{rank}(\mathcal{T})$ is equal to the smallest CPD rank r that yields a zero-error decomposition of $\mathcal{T}$, such that $\epsilon = 0$.

2.6 Conclusion

This chapter introduced tensors as higher-order generalizations of vectors and matrices, along with the concept of tensor decomposition, focusing specifically on the CP decom-

position and its implementation via CP-ALS. We also discussed the notions of tensor rank and essential uniqueness.

More generally, CP-ALS will serve as the backbone of all decoupling algorithms discussed in the following chapters.

Chapter 3

Curve Fitting

One important step of tensor decoupling algorithms involves fitting each internal function $g_1 \dots g_r$ to a set of points. More generally, the problem of finding a function that best fits a set of measurements is called curve fitting.

In this chapter, we begin by discussing polynomials as a simple but impractical solution to curve fitting problems. We then show how splines, in the form of basis splines, provide a better alternative, and how a smoothing prior can improve curve fitting with penalized splines. Finally, we discuss how the smoothing term λ can be tuned automatically via generalized cross-validation.

3.1 Curve Fitting

Curve fitting is the problem of finding a function that best fits a set of measurements. In the one-dimensional case, which is most relevant for us, we may write our *dataset* as a pair $(\mathbf{x}, \mathbf{y})$ where $\mathbf{x} \in \mathbb{R}^N$ are *data points* and $\mathbf{y} \in \mathbb{R}^N$ are *targets*. We typically assume that there exists a relationship of the form

$$y_i = f(x_i) + \epsilon_i, \quad (3.1)$$

that is, the images of the points $\mathbf{x}$ are generated by an unknown function f , but are observed with some noise ϵ^1 .

Under the assumption that such f exists, our goal is to recover it while accounting for the noise. This means finding an estimator $\hat{f}$ that satisfies

$$\hat{f}(x_i) \approx f(x_i). \quad (3.2)$$

It follows that one major challenge of curve fitting is to balance the error of our fit to the observed measurements with the *generalizability* of the estimator $\hat{f}$ to unseen data points. This is commonly called the *bias-variance tradeoff*, where fitting noise "too well" is called *overfitting*, while the opposite is called *underfitting*.

¹We typically assume that the ϵ_i terms are independent and satisfy $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

We now discuss possibilities for finding and representing the estimator $\hat{f}$ that are also common in the tensor decoupling literature.

3.2 Polynomial Interpolation

We begin by discussing (arguably) the simplest solution to the curve fitting problem: *polynomial interpolation*.

Given a vector $\mathbf{x}$ of N points, the *interpolation theorem* states that there exists a polynomial p of degree $N - 1$ that passes *exactly* through each of these points

$$\begin{aligned} p(x_1) &= y_1, \\ p(x_2) &= y_2, \\ &\vdots \\ p(x_N) &= y_N, \end{aligned} \tag{3.3}$$

meaning that the error, which is typically measured using the *residual sum of squares* (RSS), is equal to zero

$$\text{RSS} = \sum_{i=1}^N (y_i - p(x_i))^2 = 0. \tag{3.4}$$

This means that p is a *perfect fit* to our dataset, and at first glance, seems like a simple solution to our problem. There are however important issues with the polynomial approach that motivate the use of splines, introduced in the next section.

First of all, p perfectly fits our data points, and therefore would highly overfit on a set of noisy measurements, generalizing poorly to points $x \notin \mathbf{x}$.

Second, the degree of the polynomial grows linearly with the number of data points, meaning that this approach requires a polynomial of degree 99 to fit a dataset of 100 points. Solving the resulting system causes computational challenges as the *condition number*² of Vandermonde matrices grows exponentially with their size [Gautschi and Inglese, 1987].

One consequence of this is called *Runge's phenomenon* [Runge et al., 1901]: large oscillations that appear in high-degree polynomial interpolation. Figure 3.1 illustrates the phenomenon, where we can see that using higher-degree polynomials (13 and 17) leads to large oscillations near the boundaries.

Regularization only provides a partial fix.

A solution to the problems mentioned above is to introduce a *regularization term* (or to limit the degree of the polynomial). This forces the estimator towards a smoother fit, at the cost of increasing its RSS error.

²The condition number of a matrix $\mathbf{A}$ measures how sensitive the error in the solution $\mathbf{x}$ to the linear system $\mathbf{Ax} = \mathbf{y}$ is to perturbations in the data.

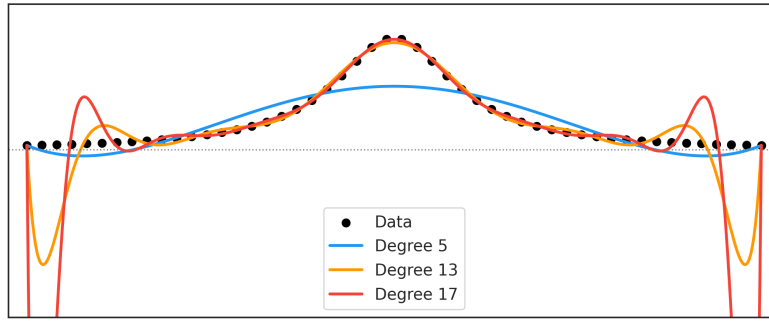


FIGURE 3.1: Runge's phenomenon in polynomial interpolation.

However, regularization does not resolve the issue of *global support*. Polynomials are defined over the entire domain, meaning that a *local* perturbation in the data influences the fitted curve *globally*. This means that adding or moving a single data point forces the entire curve to change, which is both computationally wasteful and statistically undesirable.

In summary, we would like to have a more *flexible* and numerically stable model than a single polynomial, without sacrificing quality of fit, and that provides local instead of global support.

3.3 Splines

A more flexible approach than working with a single polynomial is to partition the input space into local regions and use a set of *piecewise* low-degree polynomials, each defined over a single region.

This kind of functions are called *splines*. We often impose boundary constraints on *knots*, the points where the piecewise functions meet, to create a coherent global function. Splines are a very popular class of functions and are used in many practical applications, such as computer graphics [Piegl and Tiller, 2012] or statistical regression [Eilers and Marx, 1996].

More formally, a spline S is a piecewise polynomial function $S : [a, b] \rightarrow \mathbb{R}$ defined on an interval $[a, b]$ with certain continuity constraints. The interval is covered by m ordered sub-intervals $[t_i, t_{i+1}]$. On each of these sub-intervals, we define a polynomial $p_i : [t_i, t_{i+1}] \rightarrow \mathbb{R}$. When these polynomials are of degree d , we say that S is a spline of order $k = d + 1$. We can now define S as follows

$$S(x) = \begin{cases} p_0(x) & \text{if } t_0 \leq x \leq t_1, \\ p_1(x) & \text{if } t_1 \leq x \leq t_2, \\ \vdots & \\ p_{m-1}(x) & \text{if } t_{m-1} \leq x \leq t_m, \end{cases} \quad (3.5)$$

where the points t_i are the knots, and the vector $\mathbf{t} = [t_0 \dots t_m]$ is called *knot vector*.

To ensure smoothness, we enforce continuity constraints on neighboring polynomials, such that pairs (p_i, p_{i+1}) must share derivative values at knots up to a certain degree r

$$\begin{aligned} p_i(t_{i+1}) &= p_{i+1}(t_{i+1}), \\ p'_i(t_{i+1}) &= p'_{i+1}(t_{i+1}), \\ &\vdots \\ p_i^{(r)}(t_{i+1}) &= p_{i+1}^{(r)}(t_{i+1}). \end{aligned} \tag{3.6}$$

A spline that is continuous up to (and including) the r -th derivative is said to be C^r continuous. A common constraint used in curve fitting is to enforce equality up to the second derivatives (C^2 continuity).

3.4 Basis Splines

Basis splines, abbreviated as B-splines, serve as fundamental building blocks for creating any spline function. They are made of B-spline *basis functions* $B_1 \dots B_{df}$ and are controlled by coefficients called *control points*. The number of basis functions (and control points) df is commonly referred to as the *degrees of freedom* of the spline.

Any spline of order $k = d + 1$ can be expressed as a linear combination of basis functions

$$S(x) = \sum_{i=1}^{df} c_i B_{i,d}(x), \tag{3.7}$$

where each $B_{i,d}$ is a piecewise polynomial of degree d , and is fully defined by the knot vector $\mathbf{t}$. In fact, given knot vector and degree, basis functions can be constructed using the Cox–de Boor recursive formula [Cox, 1972, De Boor, 1972]. The base case is

$$B_{i,0}(x) = \begin{cases} 1 & \text{if } t_i \leq x \leq t_{i+1} \\ 0 & \text{otherwise} \end{cases}, \tag{3.8}$$

and for basis functions of degree $d + 1$

$$B_{i,d+1}(x) = \frac{(x - t_i) \cdot B_{i,d}(x)}{t_{i+d+1} - t_i} + \frac{(t_{i+d+2} - x) \cdot B_{i+1,d}(x)}{t_{i+d+2} - t_{i+1}}. \tag{3.9}$$

This means that in a curve fitting context, given knots and degree, we are only left with finding the coefficients $\mathbf{c}$. Those can be found by minimizing the RSS

$$\min_{\mathbf{c}} \|\mathbf{y} - \mathbf{B}\mathbf{c}\|^2, \tag{3.10}$$

where $\mathbf{B} \in \mathbb{R}^{N \times df}$ is the B-spline *design matrix*

$$\mathbf{B} = \begin{bmatrix} B_{1,d}(x_1) & B_{2,d}(x_1) & \dots & B_{df,d}(x_1) \\ B_{1,d}(x_2) & B_{2,d}(x_2) & \dots & B_{df,d}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ B_{1,d}(x_N) & B_{2,d}(x_N) & \dots & B_{df,d}(x_N) \end{bmatrix}. \tag{3.11}$$

Similarly to the closed-form solution of equation (2.16), we can easily solve equation (3.10) for the B-spline coefficients

$$\begin{aligned} 0 &= \frac{\partial}{\partial \mathbf{c}} \|\mathbf{y} - \mathbf{B}\mathbf{c}\|^2 = 2\mathbf{B}^T(\mathbf{y} - \mathbf{B}\mathbf{c}), \\ \mathbf{c} &= (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{y}. \end{aligned} \quad (3.12)$$

The procedure for fitting a B-spline to a set of points can be summarized as follows:

- (0) Choose the B-spline order $k = d + 1$ and degrees of freedom df .
- (1) Place $df - k$ *internal knots* in the spline range $[a, b]$, and construct knot vector $\mathbf{t} \in \mathbb{R}^{df+k}$ by repeating the boundary values k times at respective ends. This is required to ensure our spline interpolates the first and last control points.
- (2) Build the design matrix $\mathbf{B}$ using equation (3.9).
- (3) Find the coefficients $\mathbf{c}$ that minimize equation (3.10) using equation (3.12).
- (4) Construct the B-spline $S(x) = \sum_{i=1}^{df} c_i B_{i,d}(x)$.

The resulting $df + k$ total number of knots in step (1) comes from the fact that each basis has local support over k consecutive knot spans $[t_i, t_{i+k}]$, and indexing all df basis functions requires knot indices from t_1 to t_{df+k} , hence the need for a total of $df + k$ knots.

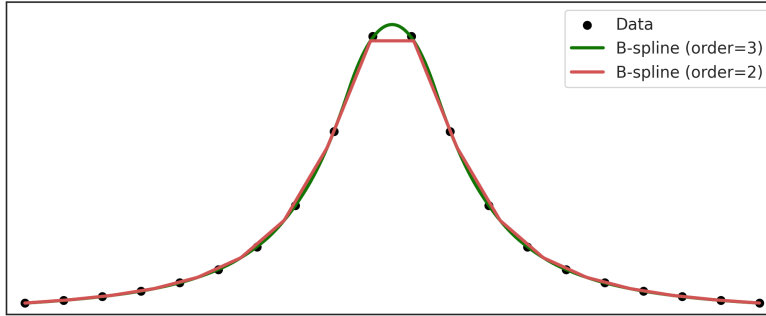


FIGURE 3.2: B-splines of order 2 and 3 applied to Runge's function.

In Figure 3.2 we can see the result of using B-splines instead of polynomials on the same set of observations. Since B-splines are made of piecewise low-degree polynomials, there is no such issue that modifying one part of the spline leads to unexpected behavior or large oscillations elsewhere. In fact, we observe that even a spline of order 2 (piecewise linear) leads to a better fit than polynomials.

A B-spline of order k has the property of being C^{k-2} continuous, therefore a widely used setting in curve fitting is to use B-splines of order 4—*cubic B-splines*—since C^2 continuity is sufficient for visually smooth curves.

3.5 Penalized Splines

One important parameter to control when fitting B-splines is the number of basis functions. Indeed, too few will likely underfit our measurements, while too many can easily lead to overfitting. In fact, choosing the right number of basis functions df is not trivial, as illustrated by the following example.

Figure 3.3 shows the impact of choosing too few or too many control points for a complex nonlinear function. With $df = 4$ (left plot), we observe that the spline clearly underfits our target, while the rightmost plot ($df = 30$) overfits heavily on the noisy measurements.

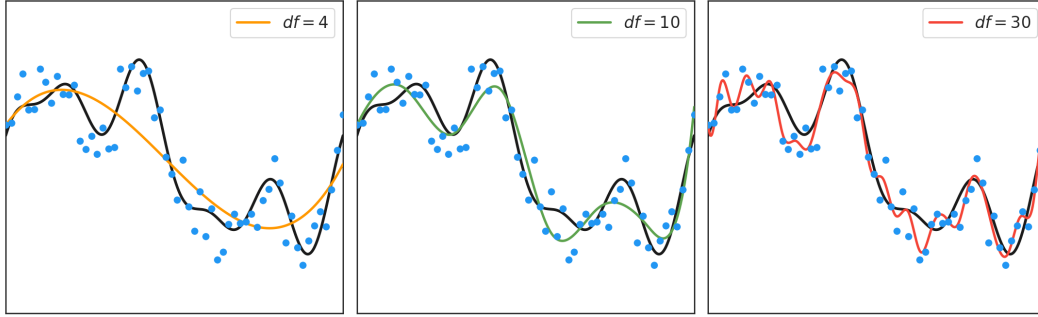


FIGURE 3.3: Importance of choosing the right number of control points with B-splines.

Penalized splines [Eilers and Marx, 1996], or P-splines, are B-splines for which the optimal coefficients [equation (3.10)] are solved with an additional smoothness penalty. P-splines only change how the objective function is formulated, however, this simple change can have a big impact, especially when using a large number of control points.

Because they are C^2 continuous, cubic B-splines are smooth by definition, however, the smoothness of P-splines is of a different kind, one that makes them particularly relevant in the context of noisy data fitting.

Penalized splines optimize the following objective

$$\min_{\mathbf{c}} \|\mathbf{y} - \mathbf{B}\mathbf{c}\|^2 + \lambda \|\mathbf{D}\mathbf{c}\|^2, \quad (3.13)$$

where $\mathbf{D}$ is a *finite difference matrix* and $\lambda \in \mathbb{R}_{\geq 0}$ controls the weight of the regularization. It follows that λ can be used to tune the smoothness of our fit, where $\lambda = 0$ leads to the B-spline setting of equation (3.10) and $\lambda \rightarrow \infty$ to a linear model (a straight line).

$\mathbf{D}$ will typically take the shape of a second-order finite difference matrix

$$\mathbf{D} = \begin{bmatrix} 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \\ 0 & \dots & 0 & 1 & -2 & 1 \end{bmatrix} \quad \mathbf{D} \in \mathbb{R}^{(df-2) \times df}, \quad (3.14)$$

although other forms are possible [Eilers and Marx, 1996]. Higher-order difference matrices can be constructed by repeatedly applying a first-order difference matrix to $\mathbf{D}$.

Furthermore, just like in B-spline regression, equation (3.13) has a closed-form solution

$$\mathbf{c} = (\mathbf{B}^T \mathbf{B} + \lambda \mathbf{D}^T \mathbf{D})^{-1} \mathbf{B}^T \mathbf{y}. \quad (3.15)$$

By acting on the coefficients, P-splines directly penalize the roughness of the curve. This simple but powerful change allows us to increase the number of control points without overfitting, provided that we properly choose the value of λ .

The simplicity of P-splines makes them particularly convenient to use compared to more sophisticated smoothing approaches, such as smoothing splines [Wahba, 1990].

At this point, however, it may seem like we have simply traded one hyperparameter (the degrees of freedom df) for another (the smoothing term λ). One advantage of using this P-spline formulation is that λ can be tuned automatically via generalized cross-validation.

3.6 Generalized Cross-Validation

A common and powerful technique for finding optimal values for hyperparameters in curve fitting problems is called *leave-one-out cross-validation* (LOO-CV). With a dataset of N points, the idea is to use $N - 1$ points to fit our model, and the remaining one for evaluation. This process is computationally expensive but produces nearly unbiased estimates. It is therefore widely used when dealing with small datasets and simple models.

When dealing with larger datasets, however, this can not be computed in reasonable time. Nevertheless, Craven and Wahba [1978] showed that for linear smoothers (a category in which P-splines fall), a good estimate of LOO-CV(λ) can be computed in a single step.

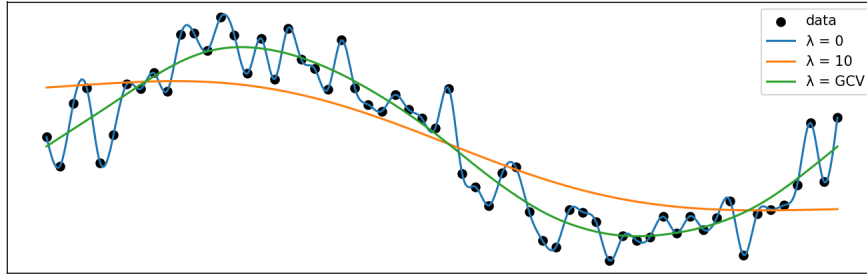


FIGURE 3.4: GCV automatically finds the λ value that best balances error and smoothness.

This method is called *generalized cross-validation* (GCV) and is computed as follows

$$\text{GCV}(\lambda) = \frac{\frac{1}{N} \|\mathbf{y} - \mathbf{B}\mathbf{c}\|^2}{\left(1 - \frac{\text{tr}(\mathbf{G})}{N}\right)^2}, \quad (3.16)$$

where $\text{tr}(\mathbf{G})$ denotes the *effective degrees of freedom* of our estimator and $\mathbf{G}$, the *hat-matrix*, is defined as

$$\mathbf{G} = \mathbf{B}(\mathbf{B}^T \mathbf{B} + \lambda \mathbf{D}^T \mathbf{D})^{-1} \mathbf{B}^T. \quad (3.17)$$

Equation (3.16) can intuitively be read as a balance between the mean RSS [equation (3.4)] in the numerator, and the *complexity* of our estimator in the denominator, as a function of hyperparameter λ . According to equation (3.16), a function that can provide the same RSS error with a simpler model is to be preferred, which is in line with our intuitions about generalization.

It follows that finding the best λ can be formulated as

$$\lambda^* \leftarrow \arg \min_{\lambda} \text{GCV}(\lambda). \quad (3.18)$$

To solve equation (3.18) in practice, we can perform a grid-search over a range of values, typically in log-space. Otherwise, one can use 1-dimensional optimization techniques such as Brent's method [Brent, 2013], or, since equation (3.16) is differentiable, Newton-type methods [Nocedal and Wright, 2006].

Figure 3.4 shows the effect of different λ values when fitting a noisy nonlinear function. We observe that with GCV, we can efficiently find the one that balances being smooth (unlike $\lambda = 0$, which corresponds to a standard B-spline) with a low RSS error (unlike $\lambda = 10$, which clearly underfits our data points).

By placing a knot at every data point, which would lead standard B-splines to largely overfit on noise as we showed in Figure 3.3, and using GCV to find the optimal λ , we can entirely remove the need to choose one hyperparameter while maximizing the expressiveness of our estimator. This will turn out to be very useful in Chapter 5.

3.7 Conclusion

In this chapter, we explored the general problem of curve fitting, that is, learning good estimators from a limited set of measurements. We showed that using polynomials was impractical and how splines provided a more convenient representation. We then discussed a widely used implementation of splines with B-splines, before diving into penalized splines and their smoothing prior.

Finally, we showed how P-splines coupled with generalized cross-validation allow to increase expressiveness while controlling for overfitting.

Chapter 4

Tensor Decoupling Algorithms

In this chapter, we describe the general decoupling methodology. We then present a selection of algorithms, explaining for each of them how it aims to improve upon previous approaches. This chapter also serves to motivate our own contributions in Chapter 5.

4.1 General Methodology

4.1.1 Problem Statement

We consider a multivariate, vector-valued target function $\mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^n$, and select a sampling range $[a, b] \subseteq \mathbb{R}$. For ease of notation and without loss of generality, we assume the same sampling range for all inputs, although it does not have to be the case in practice.

The goal of tensor decoupling methods is to find a function $\hat{\mathbf{f}} : [a, b]^m \rightarrow \mathbb{R}^n$ such that

$$\hat{\mathbf{f}}(\mathbf{x}) \approx \mathbf{f}(\mathbf{x}) \quad \forall \mathbf{x} \in [a, b]^m, \quad (4.1)$$

$$\text{subject to } \hat{\mathbf{f}}(\mathbf{x}) = \mathbf{W}\mathbf{g}(\mathbf{V}^T \mathbf{x}), \quad (4.2)$$

where $\mathbf{W} \in \mathbb{R}^{n \times r}$, $\mathbf{V} \in \mathbb{R}^{m \times r}$, $\mathbf{g} : \mathbb{R}^r \rightarrow \mathbb{R}^r$ and $r \in \mathbb{N}$ is the *rank*.

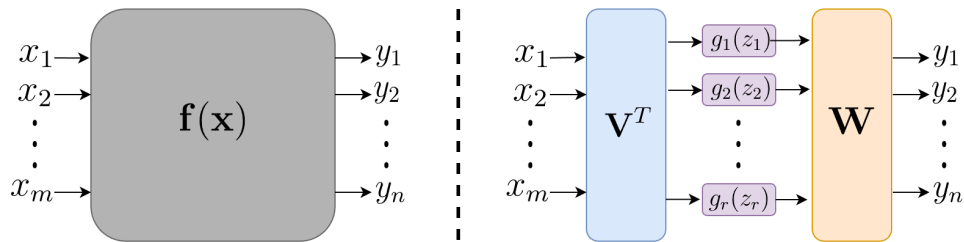


FIGURE 4.1: The tensor decoupling process aims to transform a black-box multivariate function (on the left) into a structured decoupled representation.

The rank defines the number of *branches*, which corresponds to the number of internal univariate functions

$$\mathbf{g}(u_1 \dots u_r) = [g_1(u_1) \dots g_r(u_r)]^T, \quad (4.3)$$

applied elementwise to $\mathbf{u} = \mathbf{V}^T \mathbf{x}$. The mapping from target function $\mathbf{f}$ to decoupled representation $\mathbf{W}\mathbf{g}(\mathbf{V}^T \mathbf{x})$ is illustrated in Figure 4.1.

Furthermore, we assume that $\mathbf{f}$ itself is unknown, but that we can collect output $\mathbf{y} \in \mathbb{R}^n$ and Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ for any sample $\mathbf{x} \in [a, b]^m$.

4.1.2 Methodology

If we assume that $\mathbf{f}$ satisfies the decoupled structure of equation (4.2), that is, two linear mappings around a set of univariate nonlinearities¹, we find that $\mathbf{J}$ satisfies

$$\mathbf{J} = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \mathbf{W} \operatorname{diag} \left(\left\{ \frac{\partial g_i(\mathbf{v}_i^T \mathbf{x})}{\partial \mathbf{x}} \right\} \right) \mathbf{V}^T, \quad (4.4)$$

via application of the chain rule [Dreesen et al., 2015].

If we collect N operating points $\{\mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}\}$, stack their Jacobians with respect to $\mathbf{f}$ into a 3-way tensor

$$\mathcal{J} = \operatorname{stack}(\mathbf{J}^{(1)} \dots \mathbf{J}^{(N)}) \quad \mathcal{J} \in \mathbb{R}^{n \times m \times N}, \quad (4.5)$$

and compute the rank- r CP decomposition of $\mathcal{J}$, we have that

$$\mathcal{J} \approx \sum_{i=1}^r \mathbf{w} \circ \mathbf{v} \circ \mathbf{h} \equiv \llbracket \mathbf{W}, \mathbf{V}, \mathbf{H} \rrbracket, \quad (4.6)$$

that is, the first two factors returned by the CPD directly map to equation (4.2). Additionally, under the assumption that $N \geq r$ with $\mathbf{H}$, $\mathbf{W}$, and $\mathbf{V}$ full rank, we have that Kruskal's theorem [equation (2.10)] simplifies to

$$\min(m, r) + \min(n, r) \geq r + 2, \quad (4.7)$$

which are mild conditions in practice [Dreesen et al., 2015], since it seems natural to use more operating points than internals, and factors are typically initialized randomly. This turns Kruskal's theorem into a statement that relates the number of inputs and outputs of the target to the CPD rank, essentially saying that a rank chosen improperly leads to non-unique decompositions (dealing with non-unique CPD scenarios is discussed later).

At this point, we already stated that from equation (4.4), $\mathbf{W}$ and $\mathbf{V}$ directly translate into equation (4.2), and thus, the challenge is to recover the internals $\mathbf{g}$. This is in fact a central problem in tensor decoupling, which we explore in the following sections.

¹As a side remark, a block model in the form linear $\rightarrow$ nonlinear $\rightarrow$ linear is known as a *Wiener-Hammerstein system*. More precisely, the decoupled structure of equation (4.2) is an instance of a parallel Wiener-Hammerstein system identified from Jacobian measurements.

Summary of the tensor decoupling idea.

The core "skeleton" of tensor decoupling algorithms can be described as follows:

- (1) Sample a set of N points $\{\mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}\} \in [a, b]^m$ and use them to collect information about the function $\mathbf{f}$ (outputs and Jacobians). Stack the Jacobians to form the tensor $\mathcal{J}$.
- (2) Initialize the factors $\mathbf{W}, \mathbf{V}, \mathbf{H}$ with respect to the chosen rank r .
- (3) Compute a rank- r CP decomposition $\llbracket \mathbf{W}, \mathbf{V}, \mathbf{H} \rrbracket$ of the Jacobian tensor $\mathcal{J}$.
- (4) Recover the internal univariates $g_1 \dots g_r$.
- (5) Return $(\mathbf{W}, \mathbf{V}, \mathbf{g})$.

This procedure implies that certain hyperparameters must be specified by the user, which may in turn directly impact performance and computational costs, such as:

- The rank of the decomposition which directly defines the number of internal univariates (*branches*), making it a crucial parameter.
- The number of operating points $N \in \mathbb{N}$, for certain functions, a larger number of points is required to reach a low error. It is also directly tied to computational costs, as it controls for the size of the third factor $\mathbf{H} \in \mathbb{R}^{N \times r}$.
- A maximum number of iterations $T \in \mathbb{N}$, which can impact the quality of the decoupled representation, since too few iterations will lead to a suboptimal solution.
- An assumption on the internals that allows for their recovery.

Measuring the quality of our decoupling.

To measure the quality of our decoupled representation with respect to the target, we can look at the tensor reconstruction error from returned factors [equation (2.5)]. Otherwise, another relevant metric is the *per-output relative error*, defined as

$$\epsilon_i = \frac{\sqrt{\frac{1}{N} \sum_{j=1}^N \left(f_i(\mathbf{x}_j) - \hat{f}_i(\mathbf{x}_j) \right)^2}}{\sqrt{\frac{1}{N} \sum_{j=1}^N \left(f_i(\mathbf{x}_j) - \mathbb{E}[f_i] \right)^2}}, \quad (4.8)$$

which directly measures how well our decoupling approximates the i -th output of the target, and where $100 \cdot \epsilon_i$ can be interpreted as a percentage.

Note that looking only at the mean per-output error $\frac{1}{n} \sum_{i=1}^n \epsilon_i$ can be misleading, since we typically care about minimizing *all* outputs equally, and therefore should be (at minima) coupled with a measure of variance.

4.2 Polynomial Decoupling

After describing the general idea and methodology of tensor decoupling, we explain in more detail the initial algorithm proposed by Dreesen et al. [2015].

In their work, $\mathbf{f}$ is assumed to be a multivariate polynomial where each f_i is of degree at most d . Under this assumption, we can represent our internals $\mathbf{g}$ also using degree- d polynomials

$$g_i(u) = c_{0,i} + c_{1,i}u + c_{2,i}u^2 + \cdots + c_{d,i}u^d, \quad (4.9)$$

meaning that $\mathbf{g}$ can be fully specified by a matrix of coefficients $\mathbf{C} \in \mathbb{R}^{(d+1) \times r}$, where each column $\mathbf{c}_i$ defines the i -th internal.

Algorithm 3 Polynomial tensor decoupling [Dreesen et al., 2015].

```

function POLYNOMIAL-DECOUPLING( $\mathbf{X}, \mathbf{Y}, \mathcal{J}, r, T, d, \tau$ )
    ( $\mathbf{W}, \mathbf{V}, \mathbf{H}$ )  $\leftarrow$  CP-ALS( $\mathcal{J}, r, T, \tau$ ) ▷ Algorithm 2
     $\mathbf{C} \leftarrow$  Solve-Coefficients( $\mathbf{X}, \mathbf{Y}, \mathbf{W}, \mathbf{V}, d$ )
    return ( $\mathbf{W}, \mathbf{V}, \mathbf{C}$ )
end function
    
```

The procedure of Dreesen et al. [2015] can be concisely described in Algorithm 3, where the arguments of the Polynomial-Decoupling function are described below:

- $\mathbf{X} \in \mathbb{R}^{N \times m}$ are the N samples $\{\mathbf{x}^{(1)} \dots \mathbf{x}^{(N)}\}$ stacked into a matrix.
- $\mathbf{Y} \in \mathbb{R}^{N \times n}$ are the N outputs of $\mathbf{f}$ given samples $\mathbf{X}$, stacked into a matrix.
- $\mathcal{J} \in \mathbb{R}^{n \times m \times N}$ is the Jacobian tensor defined in equation (4.5).
- $d \in \mathbb{N}$ is the degree of internal polynomials.
- r, T and τ are the same as in Algorithm 2.

Of course, after computing the CP decomposition of $\mathcal{J}$, we have to recover the coefficients of our internals. We now explain the implementation of Solve-Coefficients in Algorithm 3.

Solving for the internals.

Let $\mathbf{u} = \mathbf{V}^T \mathbf{x}$ denote the inputs to $\mathbf{g}$ in equation (4.2).

After computing the (unique) rank- r CP decomposition (recall that it directly returns $\mathbf{V}$ and $\mathbf{W}$), we have that $\mathbf{W}\mathbf{g}(\mathbf{u}) = \mathbf{y}$, which may be alternatively written as

$$\mathbf{y} = \mathbf{W} \begin{bmatrix} g_1(u_1) \\ \vdots \\ g_r(u_r) \end{bmatrix} = \mathbf{W} \begin{bmatrix} c_{0,1} + c_{1,1}u_1 + c_{2,1}u_1^2 + \cdots + c_{d,1}u_1^d \\ \vdots \\ c_{0,r} + c_{1,r}u_r + c_{2,r}u_r^2 + \cdots + c_{d,r}u_r^d \end{bmatrix}. \quad (4.10)$$

From equation (4.10), we can extract the coefficients to form a vector $\mathbf{c} \in \mathbb{R}^{r(d+1)}$, leading

to the following linear system

$$\mathbf{y} = \mathbf{W} \begin{bmatrix} 1 & u_1 & \cdots & u_1^d & & & \\ & & & & \ddots & & \\ & & & & & 1 & u_r & \cdots & u_r^d \end{bmatrix} \begin{bmatrix} c_{0,1} \\ \vdots \\ c_{d,1} \\ \vdots \\ c_{0,r} \\ \vdots \\ c_{d,r} \end{bmatrix}, \quad (4.11)$$

where empty spaces denote zeros.

By repeating this system for each of the N input-output pairs $(\mathbf{u}^{(i)}, \mathbf{y}^{(i)})$ (see Dreesen et al. [2015] for more details), we can construct a least-squares problem and solve it for $\mathbf{C}$.

Finally, we can build our decoupled representation as described in equation (4.10), and use it with any real vector $\mathbf{x} \in [a, b]^m$, effectively reaching our goal of finding a representation that satisfies equation (4.1) (if our sampled points are representative enough).

Discussion and limitations of this approach.

Let us denote as $\mathbf{Z} = \mathbf{X}\mathbf{V} \in \mathbb{R}^{N \times r}$ the matrix containing all the inputs to our internals, that is, each row of $\mathbf{Z}$ corresponds to a $\mathbf{u}$ in equation (4.10), and each column $\mathbf{z}_i$ denotes all inputs fed to the i -th internal. This procedure can be seen as finding the polynomial coefficients $\mathbf{c}_i$ that best fit the measurements $\mathbf{y}$ from data points $\mathbf{z}_i$. It is therefore a curve fitting problem under a degree- d polynomial assumption (Chapter 3).

Note that $\hat{\mathbf{f}}$ was learned using samples $\mathbf{x} \in [a, b]^m$, and as a consequence, will typically have a large error for inputs that are not in this range. Intuitively, it follows that the larger we want this range, the more difficult learning a good $\hat{\mathbf{f}}$ becomes, especially if the target is nonlinear. This holds for any of the discussed methods, as none of them is tailored to handle out-of-range inputs.

This approach in particular assumes and relies on the uniqueness of the underlying CP decomposition. Only under this uniqueness the recovered coefficients $\mathbf{C}$ truly correspond to the evaluations of $\mathbf{g}$. If this uniqueness is lost, either through noise or a rank chosen improperly, or if the polynomial assumption does not hold, this method will fail.

Finally, here we did not use the information contained in the third factor $\mathbf{H}$. We only used the decomposition to find $\mathbf{V}$ and $\mathbf{W}$, which was enough to recover $\mathbf{C}$ under our assumptions. Leveraging the gradients contained in $\mathbf{H}$ is discussed in the next section.

4.3 Constrained Polynomial Decoupling

4.3.1 Motivation

We begin this section by taking a look at factor $\mathbf{H}$ in more detail. This factor is particularly interesting because from equation (4.4) we can show that it contains the derivatives of our

internals with respect to all intermediate inputs $[\mathbf{z}_1 \dots \mathbf{z}_r]$

$$\mathbf{H} = \begin{bmatrix} \nabla g_1(\mathbf{z}_1) \\ \vdots \\ \nabla g_r(\mathbf{z}_r) \end{bmatrix}^T = \begin{bmatrix} \frac{dg_1(z_1^{(1)})}{dz_1^{(1)}} & \dots & \frac{dg_r(z_r^{(1)})}{dz_r^{(1)}} \\ \vdots & \ddots & \vdots \\ \frac{dg_1(z_1^{(N)})}{dz_1^{(N)}} & \dots & \frac{dg_r(z_r^{(N)})}{dz_r^{(N)}} \end{bmatrix}. \quad (4.12)$$

As mentioned earlier, we would like to use this information, since in theory, we could simply interpolate and integrate in those points, and directly recover the internals. This would entirely remove the need to solve a linear system like we did in equation (4.11).

The main issue if we want to do that, is that if the underlying decomposition is non-unique, there is no guarantee that each column $\mathbf{h}_i$ defines a coherent, continuous function. These columns will simply be too noisy and form clouds of points, such that recovering functions (the coefficients $\mathbf{C}$) from them is not possible. An example of this is shown in Figure 4.2.

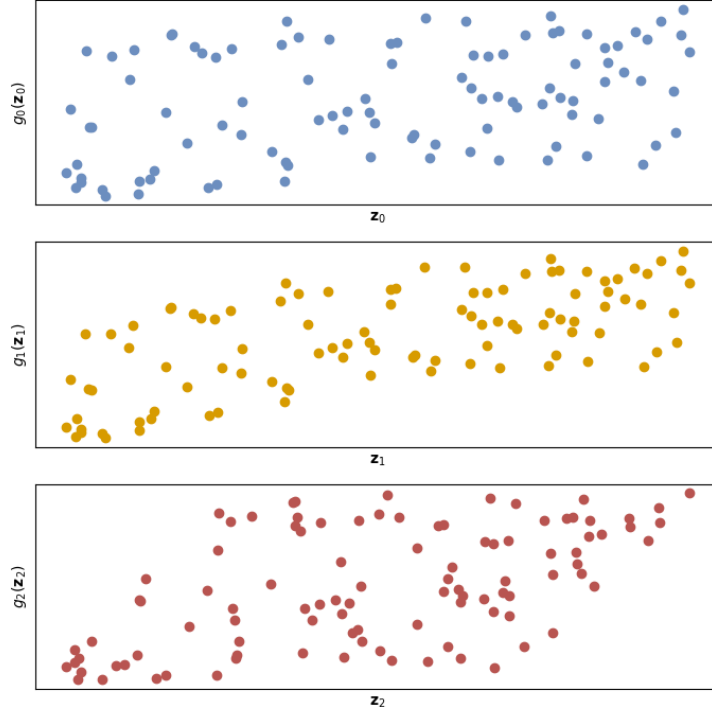


FIGURE 4.2: When a constraint is not enforced on factor $\mathbf{H}$ during the CPD, its columns are noisy and discontinuous, and hardly represent the derivatives of our internals.

This also means that a method that could "force" the columns of $\mathbf{H}$ to be coherent would allow us to deal with non-unique CPD scenarios, and still obtain sensible decoupled representations. This would have practical implications since we can not always try all possible ranks, but still want coherent and usable outputs.

One method, proposed by Hollander [2017], aims to enforce a constraint on the internals *during* the CP decomposition of $\mathcal{J}$. Indeed, if we make an assumption on their shape, for example, that they are polynomials of a certain degree, we can enforce this constraint as part of the decomposition, by forcing the points in each column $\mathbf{h}_i$ to be coherent with respect to our assumptions. More generally, methods that enforce such constraints are called *Constrained Tensor Decoupling* (CTD) algorithms.

In this approach, we still compute a CP decomposition, however, at each iteration, we directly update the coefficients of our internals such that the points in $\mathbf{H}$ represent the derivatives of a polynomial of degree $d + 1$.

Formally, the optimization problem proposed by Hollander [2017] can be written as follows

$$\begin{aligned} \min_{(\mathbf{W}, \mathbf{V}, \mathbf{H})} \quad & \|\mathcal{J} - \llbracket \mathbf{W}, \mathbf{V}, \mathbf{H} \rrbracket\|^2, \\ \text{subject to } & \mathbf{h}_i = \text{vand}(\mathbf{z}_i) \cdot \mathbf{c}'_i \text{ for } i = 1 \dots r, \end{aligned} \quad (4.13)$$

where vand builds the Vandermonde matrix of degree d

$$\text{vand}(\mathbf{z}) = \begin{bmatrix} 1 & z_1 & z_1^2 & \dots & z_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & z_N & z_N^2 & \dots & z_N^d \end{bmatrix}, \quad (4.14)$$

and $\mathbf{c}'_i \in \mathbb{R}^{d+1}$ is the vector of coefficients for the derivative polynomial g'_i .

If we use this CTD approach, we only need to integrate over g'_i , fully defined by $\mathbf{c}'_i$, and which is computed as part of the decomposition

$$c_{k,i} = \frac{c'_{k-1,i}}{k}, \quad (4.15)$$

for $k = 1 \dots d$, and where $c_{0,i}$ is the constant of integration which cannot be determined from $\mathbf{c}'_i$ alone.

Overall, this method removes the need to solve a system for the internals as a separate post-decomposition step, by giving us some pseudo-guarantees on the columns of $\mathbf{H}$.

4.3.2 Implementation

This polynomial projection only concerns the third factor $\mathbf{H}$, $\mathbf{W}$ and $\mathbf{V}$ can still be updated in standard CP-ALS fashion. To update $\mathbf{H}$, the idea is to first find the polynomial coefficients $\mathbf{C}$ that minimize

$$\min_{\mathbf{C}} \|\mathcal{J}_{(3)} - [\text{vand}(\mathbf{z}_1) \cdot \mathbf{c}_1 \dots \text{vand}(\mathbf{z}_r) \cdot \mathbf{c}_r] (\mathbf{V} \odot \mathbf{W})^T\|, \quad (4.16)$$

and then compute our updated $\mathbf{H}$ by applying equation (4.13).

The coefficients that minimize equation (4.16) can be computed as follows [Hollander, 2017]

$$\text{vec}(\mathbf{C}) \leftarrow \left(((\mathbf{V} \odot \mathbf{W}) \otimes \mathbf{I}) \begin{bmatrix} \text{vand}(\mathbf{z}_1) & & \\ & \ddots & \\ & & \text{vand}(\mathbf{z}_r) \end{bmatrix} \right)^+ \text{vec}(\mathcal{J}_{(3)}), \quad (4.17)$$

where $^+$ denotes the matrix pseudo-inverse.

Using this, we derive the procedure for computing the update of $\mathbf{H}$ in Algorithm 4.

Algorithm 4 Update of factor $\mathbf{H}$ under a degree- d polynomial constraint.

```

function CTD-UPDATE-H( $\mathcal{J}_{(3)}$ ,  $\mathbf{X}$ ,  $\mathbf{W}$ ,  $\mathbf{V}$ ,  $r$ ,  $d$ )
     $\mathbf{Z} \leftarrow \mathbf{X}\mathbf{V}$  ▷ Internals inputs.
     $\mathbf{\Lambda} \leftarrow \text{diag}(\text{vand}(\mathbf{z}_1) \dots \text{vand}(\mathbf{z}_r))$  ▷ Block diagonal matrix of equation (4.17).
     $\mathbf{c} \leftarrow (((\mathbf{V} \odot \mathbf{W}) \otimes \mathbf{I})\mathbf{\Lambda})^+ \text{vec}(\mathcal{J}_{(3)})$  ▷ Equation (4.17)
     $\mathbf{C} \leftarrow \text{Reshape}(\mathbf{c})$  ▷ The  $i$ -th column of  $\mathbf{C}$  defines the  $i$ -th internal.
    for  $i = 1 \dots r$  do
         $\mathbf{h}_i \leftarrow \text{vand}(\mathbf{z}_i) \cdot \mathbf{c}_i$  ▷ The  $i$ -th column of  $\mathbf{H}$  is a polynomial of degree  $d$ .
    end for
    return ( $\mathbf{H}$ ,  $\mathbf{C}$ )
end function

```

We now have all the elements to describe the full polynomial CTD algorithm.

Algorithm 5 Constrained polynomial tensor decoupling [Hollander, 2017].

```

function CTD-POLYNOMIAL( $\mathbf{X}$ ,  $\mathbf{Y}$ ,  $\mathcal{J}$ ,  $r$ ,  $T$ ,  $d$ ,  $\tau$ )
     $(\mathbf{W}, \mathbf{V}, \mathbf{H}) \sim \mathcal{N}(0, 1)$ 
    for  $t = 1 \dots T$  do
         $\mathbf{W} \leftarrow \text{Factor-Solve}(\mathcal{J}_{(1)}, \mathbf{V}, \mathbf{H})$  ▷ Algorithm 1
         $\mathbf{V} \leftarrow \text{Factor-Solve}(\mathcal{J}_{(2)}, \mathbf{W}, \mathbf{H})$  ▷ Algorithm 1
         $(\mathbf{H}, \mathbf{C}') \leftarrow \text{CTD-Update-H}(\mathcal{J}_{(3)}, \mathbf{X}, \mathbf{W}, \mathbf{V}, r, d)$  ▷ Algorithm 4
    end for
     $\mathbf{C} \leftarrow \text{Integrate}(\mathbf{C}')$  ▷ Equation (4.15)
    return ( $\mathbf{W}$ ,  $\mathbf{V}$ ,  $\mathbf{C}$ )
end function

```

Note that matrix $\mathbf{C}$ becomes $\mathbf{C}'$ when we view it as the output of Algorithm 4, since it contains the coefficients of the derivatives of our internals that have to be integrated before being returned. In some sense, we still have a post-processing step, but a much cheaper one.

Figure 4.3 shows the result of applying the polynomial-constrained decomposition to the same function, we observe that the points can now be easily interpolated.

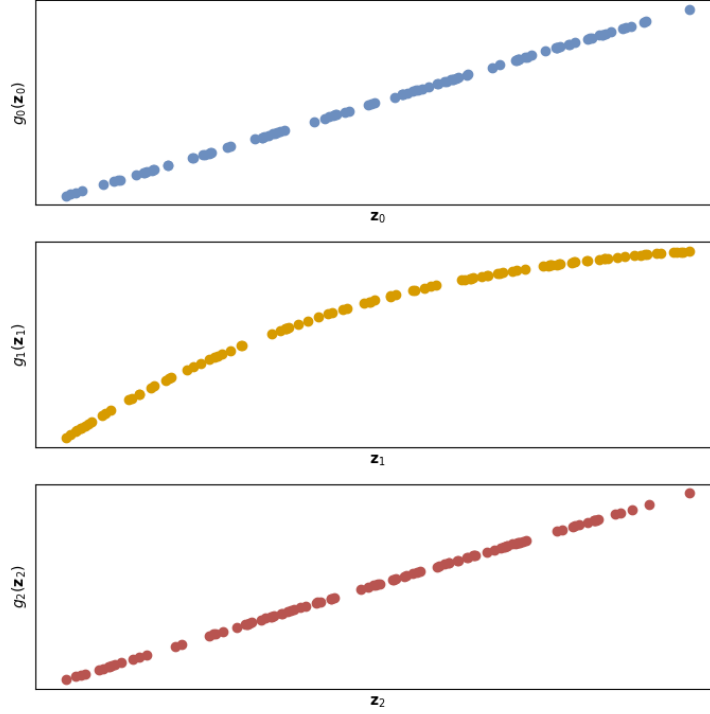


FIGURE 4.3: When a constraint is enforced during the CPD, the columns of $\mathbf{H}$ effectively represent the derivatives of our internals, and can be interpolated.

Impact on the monotonicity of CP-ALS.

One drawback of this method is that the update of $\mathbf{H}$ breaks the property that the error of the CP-ALS algorithm is always decreasing, because it introduces a dependency between $\mathbf{V}$ and $\mathbf{H}$. More specifically, given that the values in $\mathbf{H}$ depend on the values in $\mathbf{V}$, this can break monotonicity through an unfavorable $\mathbf{V}$ and an update of $\mathbf{H}$ afterwards.

In practice, this means that we can not rely on the CP-ALS convergence criterion anymore, and will typically perform T iterations and return the one with the lowest error.

4.4 Coupled Matrix-Tensor Factorization

4.4.1 Motivation

The two methods discussed so far only use the first-order information contained in $\mathcal{J}$. However, this does not allow to estimate the constant terms in the internals, which currently, should be done as a separate step.

Instead, we can leverage the information collected about the outputs of the target

$$\mathbf{Y} = [\mathbf{f}(\mathbf{x}^{(1)}) \dots \mathbf{f}(\mathbf{x}^{(N)})]^T, \quad (4.18)$$

which can alternatively be written as

$$\mathbf{Y}^T = \mathbf{W} [\mathbf{g}(\mathbf{z}^{(1)}) \dots \mathbf{g}(\mathbf{z}^{(N)})]. \quad (4.19)$$

If we introduce a new matrix $\mathbf{R} \in \mathbb{R}^{N \times r}$ and enforce $\mathbf{Y}^T = \mathbf{W} \mathbf{R}^T$, we obtain a *Coupled Matrix-Tensor Factorization* (CMTF) problem [Znied et al., 2021] for which the objective function may be written as

$$\min_{(\mathbf{W}, \mathbf{V}, \mathbf{H}, \mathbf{R})} \|\mathcal{J} - \llbracket \mathbf{W}, \mathbf{V}, \mathbf{H} \rrbracket\|^2 + \gamma \|\mathbf{Y}^T - \mathbf{W} \mathbf{R}^T\|^2. \quad (4.20)$$

Algorithms that minimize equation (4.20) have the advantage of leveraging both the first and zeroth-order information of $\mathbf{f}$, and therefore can directly estimate the constant terms of the internals, which can not be recovered from integration alone.

4.4.2 Implementation

From equation (4.20), we see that factor $\mathbf{W}$ appears in both terms of the objective, meaning that the normal CP-ALS step

$$\mathbf{W} \leftarrow \arg \min_{\mathbf{W}} \|\mathcal{J}_{(1)} - \mathbf{W}(\mathbf{H} \odot \mathbf{V})^T\| \quad (4.21)$$

becomes

$$\mathbf{W} \leftarrow \arg \min_{\mathbf{W}} \|\mathcal{J}_{(1)} - \mathbf{W}(\mathbf{H} \odot \mathbf{V})^T\|^2 + \gamma \|\mathbf{Y}^T - \mathbf{W} \mathbf{R}^T\|^2, \quad (4.22)$$

with γ typically set to a small value $0.01 \leq \gamma \leq 0.1$.

Equation (4.22) can be solved by building a stacked linear system

$$\mathbf{W} \leftarrow \arg \min_{\mathbf{W}} \left\| \begin{bmatrix} (\mathbf{H} \odot \mathbf{V}) \\ \sqrt{\gamma} \mathbf{R} \end{bmatrix} \mathbf{W}^T - \begin{bmatrix} \mathcal{J}_{(1)}^T \\ \sqrt{\gamma} \mathbf{Y} \end{bmatrix} \right\|^2. \quad (4.23)$$

Introducing $\mathbf{R}$ in equation (4.20) means that we also need to optimize for it as a separate step

$$\mathbf{R} \leftarrow \arg \min_{\mathbf{R}} \|\mathbf{Y} - \mathbf{R} \mathbf{W}^T\|^2, \quad (4.24)$$

and since $\mathbf{r}_i$ now contains the (zero-th order) evaluations of g_i , we have

$$c_{0,i} \leftarrow r_{0,i} - \hat{g}_i(z_{0,i}), \quad (4.25)$$

where $\hat{g}_i$ denotes the i -th internal obtained from integration only [equation (4.15)] and $c_{i,0}$ is the constant of integration.

Consequence for the internals.

One important consideration is with respect to the projection of the internals at each iteration. Indeed, in the previous algorithm, we were only using the columns of $\mathbf{H}$ as our measurements for fitting the internals. However, we now also want to use the zeroth-order information contained in $\mathbf{R}$.

That is, we now have the problem of fitting a function g_i such that

$$g'_i(\mathbf{z}_i) \approx \mathbf{h}_i \text{ and } g_i(\mathbf{z}_i) \approx \mathbf{r}_i, \quad (4.26)$$

in other words, both $\mathbf{H}$ and $\mathbf{R}$ must be coherent with respect to our assumptions.

Furthermore, we may *trust* the information contained in $\mathbf{H}$ more than the one in $\mathbf{R}$, since γ is typically lower than 1 in equation (4.20), this means that we want to prioritize fitting the measurements in $\mathbf{h}_i$, while allowing for a larger RSS error with respect to $\mathbf{r}_i$.

In fact we may formulate this analogously to equation (4.22)

$$g_i \leftarrow \arg \min_{g_i} \|\mathbf{h}_i - g'_i(\mathbf{z}_i)\|^2 + \gamma \|\mathbf{r}_i - g_i(\mathbf{z}_i)\|^2, \quad (4.27)$$

which again results in a stacked least-squares problem

$$g_i \leftarrow \arg \min_{g_i} \left\| \begin{bmatrix} g'_i(\mathbf{z}_i) \\ \sqrt{\gamma} g_i(\mathbf{z}_i) \end{bmatrix} - \begin{bmatrix} \mathbf{h}_i \\ \sqrt{\gamma} \mathbf{r}_i \end{bmatrix} \right\|^2. \quad (4.28)$$

Concretely, if we model each g_i as a degree- d polynomial as in the previous approaches, meaning that it can be fully specified by a coefficient vector $\mathbf{c}_i \in \mathbb{R}^{d+1}$, we have

$$\mathbf{c}_i \leftarrow \arg \min_{\mathbf{c}_i} \left\| \begin{bmatrix} \text{vand}(\mathbf{z}_i) \\ \sqrt{\gamma} \text{vand}'(\mathbf{z}_i) \end{bmatrix} \mathbf{c}_i - \begin{bmatrix} \mathbf{h}_i \\ \sqrt{\gamma} \mathbf{r}_i \end{bmatrix} \right\|^2, \quad (4.29)$$

where $\mathbf{c}_i$ now contains the constant term $c_{0,i}$ of g_i and vand' builds the derivative of the Vandermonde matrix (with an added $\mathbf{0}$ column).

Summary of the CMTF idea.

Using the zeroth-order information has the advantage of directly estimating the constant terms of our internals, as well as leading to generally better decoupled representations.

This results in a coupled matrix-tensor factorization (CMTF) problem, which introduces three important changes to tensor decoupling algorithms (additionally to a new hyperparameter $\gamma \in \mathbb{R}_{\geq 0}$):

- Modifying the CP-ALS step with respect to $\mathbf{W}$ [equation (4.22)].
- Optimizing the new matrix $\mathbf{R}$ [equation (4.24)].
- Taking into account both the first and the zeroth-order information when fitting the internals [equation (4.27)].

We will show the practical implementation of this idea in the next algorithm.

4.5 CMTF B-Spline Decoupling

The method that we now present [De Jonghe and Ishteva, 2025] combines the idea of projecting the internals at each step (CTD) with using zeroth-order information to estimate the constant terms of our internals (CMTF). Additionally, it uses a different representation to model the internal univariates.

Using splines instead of polynomials.

Rather than using polynomials to represent the internals, De Jonghe and Ishteva [2025] propose to use a B-spline projection. Splines, as discussed in Chapter 3, have a lot of nicer properties compared to working with polynomials, including local control, numerical stability, or smoothness. This makes them more suited for curve fitting problems.

This algorithm is called CMTF-BSD (for CMTF B-Spline Decoupling). Using B-splines instead of polynomials implies that we need to specify their order k , but also the number of control points (degrees of freedom) df .

Combining equation (4.13) and equation (4.20) yields a new optimization problem

$$\begin{aligned} \min_{(\mathbf{W}, \mathbf{V}, \mathbf{H}, \mathbf{R})} \quad & \|\mathcal{J} - \llbracket \mathbf{W}, \mathbf{V}, \mathbf{H} \rrbracket\|^2 + \gamma \|\mathbf{Y}^T - \mathbf{W} \mathbf{R}^T\|^2 \\ \text{subject to } & \mathbf{h}_i = \mathbf{B}'_i \cdot \mathbf{c} \text{ and } \mathbf{r}_i = \mathbf{B}_i \cdot \mathbf{c} \text{ for } i = 1 \dots r, \end{aligned} \quad (4.30)$$

where $\mathbf{B}'_i$ and $\mathbf{B}_i$ are B-spline design matrices for g'_i and g_i , respectively.

4.5.1 Implementation

For simplicity and ease of notation, we assume a cubic B-spline representation (order 4).

Rewriting equation (4.27) with a B-spline model $\mathbf{y} = \mathbf{B}\mathbf{c}$ results in

$$\min_{\mathbf{c}} \|\mathbf{h} - \mathbf{B}'\mathbf{c}\| + \gamma \|\mathbf{r} - \mathbf{B}\mathbf{c}\|^2 = \min_{\mathbf{c}} \left\| \begin{bmatrix} \mathbf{B}' \\ \sqrt{\gamma} \mathbf{B} \end{bmatrix} \mathbf{c} - \begin{bmatrix} \mathbf{h} \\ \sqrt{\gamma} \mathbf{r} \end{bmatrix} \right\|^2, \quad (4.31)$$

where $\mathbf{B} \in \mathbb{R}^{N \times (df+1)}$ is the B-spline design matrix for $\mathbf{r}_i$ with an extra column such that $\mathbf{b}_1 = \vec{1}$ and $\mathbf{B}' \in \mathbb{R}^{N \times (df+1)}$ the one for $\mathbf{h}_i$ with $\mathbf{b}'_1 = \vec{0}$ [De Jonghe and Ishteva, 2025]. The full *projection* is shown in Algorithm 6. We use the term projection since we project the columns of $\mathbf{H}$, forcing them to match our assumptions after the ALS update of $\mathbf{H}$, rather than first solving for the coefficients as we did in Algorithm 4,

Algorithm 6 B-spline projection of factor $\mathbf{H}$ in CMTF-BSD.

```

function BSPLINES-PROJECT( $\mathbf{H}, \mathbf{R}, \mathbf{X}, \mathbf{V}, df, \gamma$ )
     $\mathbf{Z} \leftarrow \mathbf{X}\mathbf{V}$  ▷ Intermediate outputs.
    for  $i = 1 \dots r$  do
         $\mathbf{t} \leftarrow \text{Determine-Knots}(\mathbf{z}_i)$  ▷ Using quantiles for example.
         $(\mathbf{B}', \mathbf{B}) \leftarrow \text{Build-Design-Matrices}(\mathbf{t}, df)$  ▷ With added columns for  $c_0$ .
         $\mathbf{c}_i \leftarrow \arg \min_{\mathbf{c}} \left\| \begin{bmatrix} \mathbf{B}' \\ \sqrt{\gamma}\mathbf{B} \end{bmatrix} \mathbf{c} - \begin{bmatrix} \mathbf{h} \\ \sqrt{\gamma}\mathbf{r} \end{bmatrix} \right\|^2$  ▷ Equation (4.31)
         $\mathbf{h}_i \leftarrow \mathbf{B}' \cdot \mathbf{c}_i$  ▷ Project the  $i$ -th column of  $\mathbf{H}$ .
         $\mathbf{r}_i \leftarrow \mathbf{B} \cdot \mathbf{c}_i$  ▷ Project the  $i$ -th column of  $\mathbf{R}$ .
    end for
     $\mathbf{C} \leftarrow \text{stack}(\mathbf{c}_1 \dots \mathbf{c}_r)$  ▷ The  $i$ -th column contains the coefs of the  $i$ -th internal.
    return  $(\mathbf{H}, \mathbf{R}, \mathbf{C})$ 
end function

```

In practice, to choose the positions of the knots $\mathbf{t}$, we can use quantiles, so that more basis functions cover denser regions with more points.

The full CMTF-BSD procedure is described in Algorithm 7.

Algorithm 7 CMTF B-Spline Decoupling [De Jonghe and Ishteva, 2025].

```

function CMTF-BSD( $\mathbf{X}, \mathbf{Y}, \mathcal{J}, r, T, df, \gamma$ )
     $(\mathbf{W}, \mathbf{V}, \mathbf{H}, \mathbf{R}) \sim \mathcal{N}(0, 1)$ 
    for  $t = 1 \dots T$  do
         $\mathbf{W} \leftarrow \arg \min_{\mathbf{W}} \left\| \begin{bmatrix} (\mathbf{H} \odot \mathbf{V}) \\ \sqrt{\gamma}\mathbf{R} \end{bmatrix} \mathbf{W}^T - \begin{bmatrix} \mathcal{J}_{(1)}^T \\ \sqrt{\gamma}\mathbf{Y} \end{bmatrix} \right\|^2$  ▷ Equation (4.23)
         $\mathbf{V} \leftarrow \text{Factor-Solve}(\mathcal{J}_{(2)}, \mathbf{W}, \mathbf{H})$  ▷ Algorithm 1
         $\mathbf{H} \leftarrow \text{Factor-Solve}(\mathcal{J}_{(3)}, \mathbf{V}, \mathbf{W})$  ▷ Algorithm 1
         $\mathbf{R} \leftarrow \arg \min_{\mathbf{R}} \|\mathbf{Y}^T - \mathbf{W}\mathbf{R}^T\|^2$  ▷ Equation (4.24)
         $(\mathbf{H}, \mathbf{R}, \mathbf{C}) \leftarrow \text{BSplines-Project}(\mathbf{H}, \mathbf{R}, \mathbf{X}, \mathbf{V}, df, \gamma)$  ▷ Algorithm 6
    end for
    return  $(\mathbf{W}, \mathbf{V}, \mathbf{C})$ 
end function

```

4.6 Conclusion

We began this chapter by introducing the general idea and methodology of tensor decoupling. We then described the first algorithm to be introduced [Dreesen et al., 2015], as well as its limitations. We improved upon this procedure by adding a constraint during the

decomposition to enforce our assumptions on the representation of our internals [Hollander, 2017]. We also added zero-th order information to better estimate the constant terms [Zniyed et al., 2021] and guide our algorithm towards better decoupled representations. Finally, we presented the CMTF-BSD algorithm, which combines the use of zeroth-order information with a B-spline projection [De Jonghe and Ishteva, 2025] instead of a polynomial one.

Chapter 5

Improved Decoupling with Scaling and Smoothing

We now describe our main contribution in the form of two improvements to current decoupling methods. We begin by motivating these changes, showing separately how they improve tensor decoupling and lead to better representations. Finally, we describe a new decoupling algorithm built from the CMTF-BSD procedure presented in Chapter 4.

5.1 Scaling

Current tensor decoupling methods can not deal with multivariate functions that have outputs and gradients with vastly different magnitudes. To fix this, we introduce a rescaling preprocessing step.

5.1.1 Motivation

To motivate the need for rescaling, consider the following periodic target function

$$\mathbf{f}^{(1)}(x_1, x_2) = \begin{bmatrix} \cos(3\pi x_1) + x_2^2 \\ -\sin(2\pi x_2) + x_1^3 \end{bmatrix}, \quad (5.1)$$

and sampling range $[0, 1]^2$. Although this function is nonlinear, it is relatively easy for the CMTF-BSD algorithm to find a good decoupled representation when using a sufficiently high df and number of samples N . In this example, we use the following hyperparameters

$$\{\text{rank}=4, \text{ niters}=50, \text{ df}=13, \text{ N}=50\},$$

and the results are shown in Table 5.1, where for each metric, we report the mean over 20 random initializations. In this table, *Output Error* refers to equation (4.8), and *Tensor Error* to equation (2.5).

TABLE 5.1: Results of applying CMTF-BSD to $\mathbf{f}^{(1)}$ (averaged over 20 random initializations).

	$f_1^{(1)}$	$f_2^{(1)}$
Output Error (in %)	3.01	2.19
Tensor Error	0.0248	

Let us now consider a scaled version of this function

$$\mathbf{f}^{(2)}(x_1, x_2) = \begin{bmatrix} 100 (\cos(3\pi x_1) + x_2^2) \\ -\sin(2\pi x_2) + x_1^3 \end{bmatrix}, \quad (5.2)$$

which is a simple multiplication of the first output of $\mathbf{f}^{(1)}$ by a factor of 100.

This does not change the *shape* of the function and, if we were able to learn the function defined in equation (5.1) we should, in theory, also be able to do it here. However, if we run our algorithm again (with the same sampling range), we observe in Table 5.2 that the first output gets relatively well approximated (7% of error), but that the second output error explodes to more than 50%.

 TABLE 5.2: Results of applying CMTF-BSD to $\mathbf{f}^{(2)}$ (averaged over 20 random initializations).

	$f_1^{(2)}$	$f_2^{(2)}$
Output Error (in %)	7.66	53.10
Tensor Error	0.0606	

In this section, we aim to investigate why this occurs and how to fix this issue, since we would naturally like to be able to decouple any multivariate function regardless of the magnitude of its outputs.

Consequence for the least-squares problem.

More generally, multiplying the i -th output of a function by some factor α , as we did it in equation (5.2), directly affects its Jacobian

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{f}(\mathbf{x})^T \text{diag}(\dots \alpha \dots)) \equiv \mathbf{J} \cdot \text{diag}(\dots \alpha \dots), \quad (5.3)$$

where $(\dots \alpha \dots)$ denotes a zero vector with the value α at position i .

This means that two outputs with different magnitudes will also typically have gradients in $\mathcal{J}$ with different magnitudes. This is an issue considering how the factors are being optimized.

Indeed, recall that the backbone of tensor decoupling methods is a CP-ALS decomposition, which minimizes an alternating least-squares problem. For example, consider the first step of the loop (Algorithm 2), where we optimize for $\mathbf{W}$

$$\min_{\mathbf{W}} \|\mathcal{J}_{(1)} - \mathbf{W}(\mathbf{V} \odot \mathbf{H})^T\|^2. \quad (5.4)$$

We can see that the entire term, which involves comparing two matrices, gets collapsed into a single scalar error, meaning that when the gradient values in $\mathcal{J}$ are of different magnitudes, an equal relative error contributes (exponentially) differently to the final value. Here, a consequence of this is that $\mathbf{W}$ gets optimized mainly for the larger terms.

To illustrate this, imagine that we are minimizing a one-dimensional least-squares problem with 3 measurements $\mathbf{y} = [10 \ 100 \ 1000]$. Here, a relative error of 10% on each output will lead to the final error being equal to $\epsilon = 1^2 + 10^2 + 100^2$. It follows that the best way to minimize the final loss here is to optimize primarily for the third point, even if that means increasing the relative error of the others.

When we are looking to learn functions rather than just fitting data, we typically care about each output equally. Thus, we would rather have a small-but-nonzero error for each output, rather than some perfect outputs and our model being completely wrong on others.

We now present two ways to tackle this issue, presented in the
chronological order in which they have been explored.

5.1.2 Function Scaling

One way to fix this problem is to operate on a rescaled function that keeps the same shape as the target, but outputs values of comparable magnitudes.

Recall that in a decoupling setting, we have no information on the intrinsics of the target function. However, we can empirically estimate the scaling factors from our input-output pairs. This allows us to create a new target function

$$\mathbf{F}(\mathbf{x}) = \mathbf{f}(\mathbf{x})^T \text{diag}(\sigma_1 \dots \sigma_n). \quad (5.5)$$

If we assume that the outputs of $\mathbf{f}$ are in the form of equation (5.2), meaning that the scaling can be captured by a single scalar, and that each factor σ_i is chosen properly, the outputs of $\mathbf{F}$ will have similar magnitudes.

Alternatively, if we already have outputs $\mathbf{Y}$ and tensor $\mathcal{J}$, equation (5.5) is equivalent to estimating the scaling factors from $\mathbf{Y}$

$$\begin{aligned} \tilde{\mathbf{Y}} &\leftarrow \mathbf{Y} \cdot \text{diag}(\sigma_1 \dots \sigma_n), \\ \tilde{\mathcal{J}}_{i::} &\leftarrow \mathcal{J}_{i::} \cdot \text{diag}(\sigma_i), \end{aligned} \quad (5.6)$$

and using $\tilde{\mathbf{Y}}$ and $\tilde{\mathcal{J}}$ to compute the decoupling.

The challenge is therefore to choose the factors σ_i such that

$$\|\tilde{\mathbf{y}}_1\| \approx \dots \approx \|\tilde{\mathbf{y}}_n\|, \quad (5.7)$$

that is, the outputs $\mathbf{F}$ have similar magnitudes.

Choosing the scaling factors.

There are actually many possibilities for choosing the scaling factors σ_i , and we may take inspiration from classical data preprocessing techniques. Dataset rescaling and normalization are recurring themes in machine learning and data sciences, for example, when measurements are taken in different units.

For example, one could use

$$\sigma_i^{-1} = \max(\mathbf{y}_i) - \min(\mathbf{y}_i), \quad (5.8)$$

which is directly analogous to a *min-max normalization*, remapping our outputs near the range $[0, 1]$. Note that here we omit the subtraction of $\min(\mathbf{y}_i)$ in the numerator, as it is not translated in the derivative.

Another natural choice is to divide each output by its standard deviation

$$\sigma_i^{-1} = \text{std}(\mathbf{y}_i) = \sqrt{\frac{1}{N} \sum_{i=1}^N y_i^2}, \quad (5.9)$$

which this time can be related to a *standardization* procedure (or a *z-score normalization*) without the mean subtraction.

This is to show that factors can take many forms, and in practice, one can simply try many of them, since this procedure is not computationally expensive. We can get better estimates for the σ values by using more than N samples, to reduce bias due to a small N . This adds some overhead, depending on how unbiased we want our scaling factors.

Using scaling as a preprocessing step before the decoupling necessarily adds an additional step after the decoupled representation of $\mathbf{F}$ is found (let us denote it as $\hat{\mathbf{F}}$), since it must be unscaled to be compared to the initial target $\mathbf{f}$. This can be easily done as follows

$$\mathbf{f}(\mathbf{x}) \approx \hat{\mathbf{f}}(\mathbf{x}) = \hat{\mathbf{F}}(\mathbf{x}) \cdot \text{diag}(\sigma_1^{-1} \dots \sigma_n^{-1}). \quad (5.10)$$

Summary of the *function scaling* approach.

We may now summarize this preprocessing methodology:

- (1) Collect information $\mathbf{Y}$ and $\mathcal{J}$ from $\mathbf{f}$.
- (2) Estimate output scaling factors $\sigma_1 \dots \sigma_n$ from $\mathbf{Y}$ using min-max [equation (5.8)], standard deviation [equation (5.9)] or other.
- (3) Rescale $\mathbf{Y}$ and $\mathcal{J}$ according to equation (5.6).
- (4) Run the decoupling algorithm using the rescaled $\tilde{\mathbf{Y}}$ and $\tilde{\mathcal{J}}$, and obtain a decoupled representation $\hat{\mathbf{F}}$ such that $\hat{\mathbf{F}}(\mathbf{x}_i) \approx \tilde{\mathbf{y}}_i$.
- (5) Unscale $\hat{\mathbf{F}}$ to recover the initial target $\mathbf{f}(\mathbf{x})$ using equation (5.10).

Limitations.

The function scaling approach described above does not *guarantee* that the entries of the Jacobian tensor will be brought into comparable ranges.

Specifically, the scaling factors σ_i are estimated from the output values $\mathbf{Y}$ alone, with the implicit assumption that output magnitude differences are representative of gradient magnitude differences. While this holds when the scaling is *uniform* across the input domain, it may fail for functions whose outputs and gradients behave differently across regions of the input space.

We now present a more principled (and perhaps more intuitive)
approach with a direct scaling of the input Jacobian tensor.

5.1.3 Tensor Scaling

The key to finding the correct rescaling is to think about what we are really looking for, and what we mean by "the gradients of $\mathbf{f}$ should have similar magnitudes". We can be more precise by saying that *the norm of the gradients with respect to each output should be equal*.

In fact, this can be directly satisfied by requiring equality on the norms of the horizontal slices of the input Jacobian tensor $\mathcal{J}$

$$\|\mathcal{J}_{1::}\| = \dots = \|\mathcal{J}_{n::}\|. \quad (5.11)$$

Indeed, recall that each horizontal slice $\mathcal{J}_{i::} \in \mathbb{R}^{m \times N}$ is of the form

$$\mathcal{J}_{i::} = \begin{bmatrix} \frac{\partial f_i(\mathbf{x}^{(1)})}{\partial x_1^{(1)}} & \dots & \frac{\partial f_i(\mathbf{x}^{(N)})}{\partial x_1^{(N)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_i(\mathbf{x}^{(1)})}{\partial x_m^{(1)}} & \dots & \frac{\partial f_i(\mathbf{x}^{(N)})}{\partial x_m^{(N)}} \end{bmatrix}, \quad (5.12)$$

that is, the i -th slice of $\mathcal{J}$ contains the partial derivatives of the i -th output of $\mathbf{f}$ with respect to all operating points.

From equation (5.11), it follows that a natural rescaling can be easily derived

$$\tilde{\mathcal{J}}_{i::} \leftarrow \frac{\sqrt{mN}}{\|\mathcal{J}_{i::}\|} \cdot \mathcal{J}_{i::}, \quad (5.13)$$

where the factor $\sqrt{mN}$ ensures that the entries do not vanish as the number of data points grow.

We now have that equation (5.11) holds, meaning that we effectively reached our initial goal of making the algorithm independent of the magnitudes of the partial derivatives of the target function.

One important point here is to not forget the relation between the target function and its Jacobian, meaning that if we rescale the derivatives using equation (5.13), we must also apply the same operation to the zero-th order data $\mathbf{Y}$

$$\tilde{\mathbf{y}}_i \leftarrow \frac{\sqrt{mN}}{\|\mathcal{J}_{i::}\|} \cdot \mathbf{y}_i. \quad (5.14)$$

Finally, just as we needed to *unscale* the learned function $\hat{\mathbf{F}}$ after running our algorithm with the function scaling approach, we must again post-process our decoupled representation before we can compare it to our target

$$\mathbf{f}(\mathbf{x}) \approx \hat{\mathbf{F}}(\mathbf{x}) \cdot \text{diag} \left(\frac{\|\mathcal{J}_{1::}\|}{\sqrt{mN}} \dots \frac{\|\mathcal{J}_{n::}\|}{\sqrt{mN}} \right). \quad (5.15)$$

The *tensor scaling* approach gives us a simple, mathematically exact, and computationally cheap pre-processing step. It is described in Algorithm 8. It is to be preferred to function scaling because of the previously discussed limitations of the latter.

Algorithm 8 Tensor scaling preprocessing.

```

function SCALE-TENSOR( $\mathcal{J} \in \mathbb{R}^{n \times m \times N}$ ,  $\mathbf{Y} \in \mathbb{R}^{N \times n}$ )
  for  $i = 1 \dots n$  do
     $\sigma_i \leftarrow \frac{\sqrt{mN}}{\|\mathcal{J}_{i::}\|}$  ▷ Equation (5.13)
     $\tilde{\mathcal{J}}_{i::} \leftarrow \sigma_i \cdot \mathcal{J}_{i::}$ 
     $\tilde{\mathbf{y}}_i \leftarrow \sigma_i \cdot \mathbf{y}_i$  ▷ Equation (5.14)
  end for
   $\boldsymbol{\sigma} \leftarrow [\sigma_1 \dots \sigma_n]$ 
  return  $(\tilde{\mathcal{J}}, \tilde{\mathbf{Y}}, \boldsymbol{\sigma})$  ▷ Vector  $\boldsymbol{\sigma}$  is later used to unscale decoupling outputs.
end function
    
```

5.1.4 Results

To illustrate the usefulness of scaling as a preprocessing step, we show in Table 5.3 that the two presented approaches are effectively able to decouple $\mathbf{f}^{(2)}$, and tensor decoupling yields the best results overall.

TABLE 5.3: Results of decoupling $\mathbf{f}^{(1)}$ and $\mathbf{f}^{(2)}$, comparing the two scaling approaches. Metrics averaged over 20 random initializations.

Scaling	None				Function				Tensor			
Function	$f_1^{(1)}$	$f_2^{(1)}$	$f_1^{(2)}$	$f_2^{(2)}$	$f_1^{(1)}$	$f_2^{(1)}$	$f_1^{(2)}$	$f_2^{(2)}$	$f_1^{(1)}$	$f_2^{(1)}$	$f_1^{(2)}$	$f_2^{(2)}$
Output Error	3.01	2.19	7.67	53.10	3.17	2.30	2.41	1.28	3.84	2.04	1.67	0.94
Tensor Error	0.0249		0.0606		0.0265		0.0227		0.0368		0.0187	

We also observe that scaling does not affect performance negatively in the $\mathbf{f}^{(1)}$ case, showing that the discussed procedures can *always* be applied as a general preprocessing step (even when the target does not a priori require it for a good representation to be found).

5.2 Smoothing

With CMTF-BSD (Algorithm 7), we use basis splines to represent our internals. However, in Chapter 3, we showed that choosing the number of control points correctly was particularly important in B-spline curve fitting, and that choosing too many or too few degrees of freedom could easily lead to over or underfitting.

This makes df a crucial parameter in CMTF-BSD, which must be tuned manually by the user, especially for complex nonlinear functions. Replacing the basis spline projection by a smooth penalized spline projection could, in theory, remove the need to choose df , since it could be set by default to a high value close or equal to N .

5.2.1 Implementation

Compared to the curve fitting problem of Chapter 3, in a CMTF algorithm, we must fit *both* first and zeroth-order information. Building on top of the B-spline projection of CMTF-BSD, we have that equation (4.31) becomes

$$\min_{\mathbf{c}} \|\mathbf{h} - \mathbf{B}\mathbf{c}\|^2 + \gamma \|\mathbf{r} - \mathbf{B}'\mathbf{c}\|^2 + \lambda \|\mathbf{D}\mathbf{c}\|^2 \quad (5.16)$$

where $\mathbf{B}'$ and $\mathbf{B}$ are the B-spline design matrices, and $\mathbf{D}$ is the P-spline finite difference matrix. In this work we assume $\mathbf{D}$ to be of second order, as we found that it gave good results in practice. Once again, equation (5.16) is solved by building a stacked least-squares problem.

After finding all B-spline coefficients $\mathbf{C}$, we can project $\mathbf{H}$ and $\mathbf{R}$ the same way that we did in Algorithm 6. The challenge is therefore to find the best value of λ in equation (5.16). Similarly to what we did in Chapter 3, we will use generalized cross-validation.

Tuning λ via generalized cross-validation.

The core step when using P-splines as our internal representations is finding a good approximation of $\lambda^* = \min_{\lambda} \text{GCV}(\lambda)$ where GCV was defined in equation (3.16).

In this CMTF setting, the expression for the hat-matrix [equation (3.17)] becomes

$$\mathbf{G} = \hat{\mathbf{B}}(\hat{\mathbf{B}}^T \hat{\mathbf{B}} + \lambda \mathbf{D}^T \mathbf{D})^{-1} \hat{\mathbf{B}}, \quad (5.17)$$

where $\hat{\mathbf{B}} = \begin{bmatrix} \mathbf{B}' \\ \sqrt{\gamma} \mathbf{B} \end{bmatrix}$ are the concatenated B-spline design matrices.

Using equation (5.17), we derive the GCV-Score function (Algorithm 9), which we can now plug into an optimization algorithm, or use as part of a grid search, to find λ^* .

Algorithm 9 Generalized cross-validation score function for a given λ .

function GCV-SCORE($\lambda, B, B', \mathbf{h}, \mathbf{r}, D, \gamma$)

$$\hat{B} \leftarrow \begin{bmatrix} B' \\ \sqrt{\gamma} B \end{bmatrix}$$

$$\mathbf{y} \leftarrow \begin{bmatrix} \mathbf{h} \\ \sqrt{\gamma} \mathbf{r} \end{bmatrix}$$

$$G \leftarrow \hat{B}(\hat{B}^T \hat{B} + \lambda D^T D)^{-1} \hat{B}$$

$$\mathbf{c} \leftarrow \arg \min_{\mathbf{c}} \left\| \begin{bmatrix} \hat{B} \\ \sqrt{\lambda} D \end{bmatrix} - \begin{bmatrix} \mathbf{y} \\ \mathbf{0} \end{bmatrix} \right\|^2$$

$$\text{RSS} \leftarrow \left\| \mathbf{y} - \hat{B} \mathbf{c} \right\|^2$$

$$\text{SCORE} \leftarrow \frac{1}{N} \text{RSS} / \left(1 - \frac{1}{N} \text{tr}(G)\right)^2$$

return SCORE

end function

Note that in practice, it is more beneficial to search for λ in logarithmic space since it restricts the search space to positive values and eases the optimization process. Optimizing for Algorithm 9 is computationally expensive, especially if we start from scratch at each iteration. In that case, we will have to compute the generalized cross-validation a total of $s \cdot r \cdot T$ times, where s denotes the number of steps taken by our optimization algorithm.

A more efficient approach (let $\lambda_i^{(t)}$ be the smoothing term for internal i at iteration t) is to assume that the optimal $\lambda_i^{(t)}$ is not very far in the loss landscape from $\lambda_i^{(t-1)}$, and use the latter as a starting point. This allows to reduce the number of optimization steps required to reach a good $\lambda_i^{(t)}$ value, by not re-starting from scratch at each iteration. In that case, we only do an extensive search at the first iteration.

Penalized splines projection.

Analogously to the B-splines projection of CMTF-BSD (Algorithm 6), we can now derive the algorithm for the penalized splines projection. Since we already defined the score function for the GCV (Algorithm 9), adding the rest is quite straightforward.

The full P-splines projection is described in Algorithm 10. For the optimization of λ , we use a Minimize placeholder function but do not constrain the implementation to a specific optimizer. Minimize takes as argument the previous λ term, which, as mentioned earlier, can be used to speed up the search process.

This projection is the key for writing an algorithm that is non-parametric with respect to the number of spline control points.

Algorithm 10 Penalized spline projection of factor $\mathbf{H}$ with GCV.

function PSPLINES-PROJECT($\mathbf{H}, \mathbf{R}, \mathbf{X}, \mathbf{V}, df, \gamma, \boldsymbol{\lambda}^{(t-1)}$)

$\mathbf{Z} \leftarrow \mathbf{X}\mathbf{V}$ ▷ Internals inputs.

$\mathbf{D} \leftarrow \text{Finite-Difference-Matrix}(df)$

for $i = 1 \dots r$ **do**

$\mathbf{t} \leftarrow \text{Determine-Knots}(\mathbf{z}_i)$ ▷ Here knots can be evenly spaced.

$(\mathbf{B}', \mathbf{B}) \leftarrow \text{Build-Design-Matrices}(\mathbf{t}, df)$ ▷ With added column for c_0 .

$\lambda_i^{(t)} \leftarrow \text{Minimize}(\text{GCV-Score}(\cdot, \mathbf{B}', \mathbf{B}, \mathbf{h}_i, \mathbf{r}_i, \mathbf{D}, \gamma), \lambda_i^{(t-1)})$

$\mathbf{c}_i \leftarrow \arg \min_{\mathbf{c}} \left\| \begin{bmatrix} \tilde{\mathbf{B}} \\ \sqrt{\lambda_i^{(t)}} \mathbf{D} \end{bmatrix} \mathbf{c} - \begin{bmatrix} \tilde{\mathbf{y}} \\ \mathbf{0} \end{bmatrix} \right\|^2$ ▷ Solve for the P-spline coefs.

$\mathbf{h}_i \leftarrow \mathbf{B}' \cdot \mathbf{c}_i$ ▷ Project the i -th column of $\mathbf{H}$.

$\mathbf{r}_i \leftarrow \mathbf{B} \cdot \mathbf{c}_i$ ▷ Project the i -th column of $\mathbf{R}$.

end for

$\mathbf{C} \leftarrow \text{stack}(\mathbf{c}_1 \dots \mathbf{c}_r)$ ▷ The i -th column contains the coefs of the i -th internal.

$\boldsymbol{\lambda}^{(t)} \leftarrow [\lambda_1^{(t)} \dots \lambda_r^{(t)}]$

return $(\mathbf{H}, \mathbf{R}, \mathbf{C}, \boldsymbol{\lambda}^{(t)})$ ▷ Return $\boldsymbol{\lambda}^{(t)}$ to speedup the search at $t + 1$.

end function

5.2.2 Results

We now compare the B-spline projection (Algorithm 6) to the P-spline projection with generalized cross-validation (Algorithm 10). More specifically, we measure their per-output accuracy [equation (4.8)] as df is increased up to the number of data points N .

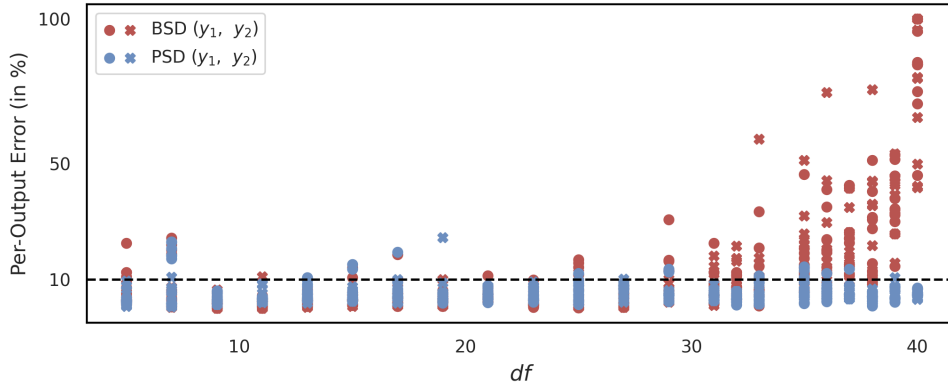


FIGURE 5.1: Per-output error as a function of df . As the number of basis functions increases, B-splines tend to overfit while the P-spline projection stays stable and consistent.

We consider a simple polynomial target $\mathbf{f} : \mathbb{R}^3 \rightarrow \mathbb{R}^2$

$$\mathbf{f}(x_1, x_2, x_3) = \begin{bmatrix} x_1^3 - 2x_1^2 + x_1x_3 \\ x_2^3 + x_3^2 + x_1x_2 \end{bmatrix} \quad (5.18)$$

and compute our decoupling with the following settings

$$\{\text{rank}=5, \text{ niters}=100, \text{ df}=[5 \dots 40], N=40\},$$

which allows us to evaluate performance as a function of df . For each number of basis, the decoupling with each projection is computed with 10 different random seeds. At each iteration of the P-spline projection the λ term is tuned with a grid-search over 100 candidates. The results are shown in Figure 5.1.

We observe that as $df \rightarrow N$, B-splines become more and more unstable (they overfit), while P-splines yield their best performance, a consequence of their ability to smooth their own coefficients. In fact, from this observation, we can derive that when using smoothing, df can be set to N by default, effectively making it a non-parameter.

Comparison to other non-parametric approaches.

The smoothing approach described here is an extension of the work of De Jonghe and Ishteva [2025], which already reduces parametric assumptions compared to using simpler internal representations (such as polynomials). The P-spline methodology explored in this work goes one step further, and turns the df hyperparameter into a purely *computational* choice, rather than a modelling one.

Another approach worth mentioning is the one of Decuyper et al. [2021], where smoothness is enforced through finite difference filters rather than a parametric representation. This is appealing and, in some sense, can be seen as truly non-parametric, since in our case, we still assume a spline model behind each internal.

Compared to this method, our approach has the advantage of being conceptually simpler, and leveraging both first and zero-th order information, which allows us to directly estimate the constant terms. Our method computes factors and internals all at once, meaning that one informs the other during the optimization, while the approach of Decuyper et al. [2021] is more modular.

5.3 Algorithm (CMTF-PSD)

Using the two improvements to CMTF-BSD described in previous sections, namely *scaling* the Jacobian tensor (Algorithm 8) and *smoothing* the internals (Algorithm 10), we now derive a new tensor decoupling algorithm, CMTF-PSD (for CMTF P-Spline Decoupling), which incorporates both changes.

Our method is described in Algorithm 11. We removed the df parameter from the list of arguments and instead directly pass N to PSplines-Project (Algorithm 10).

Algorithm 11 CMTF P-spline decoupling with tensor scaling.

```

function CMTF-PSD( $\mathbf{X}, \mathbf{Y}, \mathcal{J}, r, T, \gamma$ )           ▷ Parameter  $df$  can now be omitted.
  ( $\mathbf{W}, \mathbf{V}, \mathbf{H}, \mathbf{R}$ )  $\sim \mathcal{N}(0, 1)$                      ▷ Initialize CMTF factors.
  ( $\tilde{\mathcal{J}}, \tilde{\mathbf{Y}}, \boldsymbol{\sigma}$ )  $\leftarrow$  Scale-Tensor( $\mathcal{J}, \mathbf{Y}$ )           ▷ Algorithm 8
  for  $t = 1 \dots T$  do
     $\mathbf{W} \leftarrow \arg \min_{\mathbf{W}} \left\| \begin{bmatrix} (\mathbf{H} \odot \mathbf{V}) \\ \sqrt{\gamma} \mathbf{R} \end{bmatrix} \mathbf{W}^T - \begin{bmatrix} \tilde{\mathcal{J}}_{(1)}^T \\ \sqrt{\gamma} \tilde{\mathbf{Y}} \end{bmatrix} \right\|^2$            ▷ Equation (4.23)
     $\mathbf{V} \leftarrow$  Factor-Solve( $\tilde{\mathcal{J}}_{(2)}, \mathbf{W}, \mathbf{H}$ )           ▷ Algorithm 1
     $\mathbf{H} \leftarrow$  Factor-Solve( $\tilde{\mathcal{J}}_{(3)}, \mathbf{V}, \mathbf{W}$ )           ▷ Algorithm 1
     $\mathbf{R} \leftarrow \arg \min_{\mathbf{R}} \left\| \tilde{\mathbf{Y}}^T - \mathbf{W} \mathbf{R}^T \right\|^2$            ▷ Equation (4.24)
    ( $\mathbf{H}, \mathbf{R}, \mathbf{C}, \boldsymbol{\lambda}^{(t)}$ )  $\leftarrow$  PSplines-Project( $\mathbf{H}, \mathbf{R}, \mathbf{X}, \mathbf{V}, N, \gamma, \boldsymbol{\lambda}^{(t-1)}$ )
  end for
  return ( $\mathbf{W}, \mathbf{V}, \mathbf{C}, \boldsymbol{\sigma}$ )           ▷ Scaling factors  $\boldsymbol{\sigma}$  must be returned for inference.
end function

```

5.4 Conclusion

In this chapter, we motivated and derived two improvements to current tensor decoupling algorithms. We showed empirical results for each before combining them into a new, unified algorithm, called CMTF-PSD.

More generally, this algorithm is part of the broader objective of developing non-parametric tensor decoupling algorithms, with the goal of making these methods less dependent on the choice of their hyperparameters, and therefore, easier to use.

Chapter 6

Application to Neural Network Compression

We now focus on a practical application of the methods described earlier. Rather than trying to give a survey of all possible applications of tensor-based decoupling, we discuss one of them in particular: neural network compression. More specifically, we show that the algorithm introduced in the previous chapter can be used to effectively compress a large model by an order of magnitude, while retaining most of its performance.

6.1 Motivation

Deep learning [LeCun et al., 2015] is a subset of machine learning (learning from data) that uses deep neural networks as function approximators. These models are typically trained using first-order gradient-based optimizers (usually variants of stochastic gradient descent [Rumelhart et al., 1986, Bottou, 2010]). This means that these models, which are multivariate vector-valued functions, must be differentiable. This makes deep learning a field particularly suited for tensor decoupling algorithms.

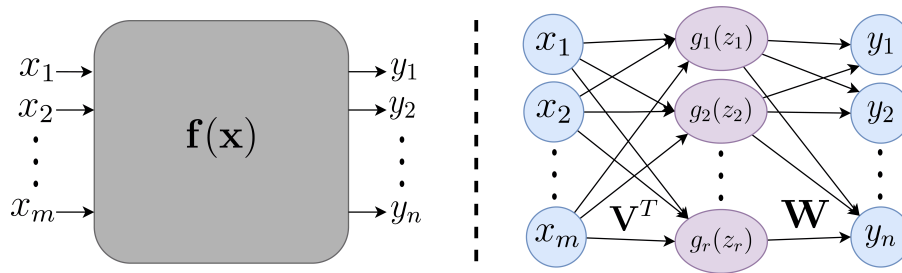


FIGURE 6.1: Tensor decoupling can be seen as mapping a black-box multivariate function to a one hidden layer neural network with flexible activations and no bias terms.

Typically, neural networks use fixed nonlinear activation functions, such as hyperbolic tangent, logistic [Rumelhart et al., 1986, LeCun et al., 2002b] or rectified linear unit (ReLU) [Nair and Hinton, 2010]. However, the decoupled representation [equation (4.2)] returned

by tensor decoupling methods uses variable nonlinearities $g_i(x) \neq g_j(x)$ for $i \neq j$.

The advantage of having fixed and identical activations is that they can be easily parallelized in a single-instruction-multiple-data (SIMD) fashion, especially on modern parallel processors, such as Graphics Processing Units (GPUs). On the other hand, it is easy to imagine that using more complex and/or learnable nonlinearities may allow us to represent the same function using fewer parameters.

This approach has been recently explored with architectures that aim to learn activation functions rather than weights, such as Kolmogorov-Arnold Networks (KANs) [Liu et al., 2025]. The idea that we present below can be seen as mapping a multilayer perceptron to a KAN-like representation, using spline nonlinearities, although we will also retain learnable weights on the edges. In fact, after computing our decoupled representation, we should be able to choose between training (or conversely, *freezing*) weights, activations, or both.

The first step in using tensor decoupling methods to compress neural networks is to select a certain layer or group of layers (or even the entire network). Then, if this subnetwork uses a total of h fixed activations, we aim to learn a decoupled representation that uses $r \ll h$ hidden nodes. The challenge, of course, is that we also wish to retain most of the performance of the larger model.

Since we control the number of nonlinear internals by setting the rank r , we can also steer the compression ratio $\frac{|\theta_{\text{target}}|}{|\theta_{\text{compressed}}|}$ where θ denotes the parameters of a model. In some sense, this can be compared to using decompositions to compress large tensors (Chapter 2), but applied to deep neural networks instead.

Compared to other approaches for compressing large deep learning models, such as knowledge distillation [Hinton et al., 2015], this approach is computationally cheaper, since it doesn't require entirely retraining another (albeit smaller) neural network.

6.2 Experiment

We train a neural network on the well-known MNIST dataset [LeCun et al., 2002a], which is the supervised classification task of mapping images of handwritten digits to their label. We used a multilayer perceptron $\mathbf{f}$ defined as

$$\mathbf{f}(\mathbf{x}) = \sigma(\mathbf{x}^T \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (6.1)$$

with $\mathbf{W}_1 \in \mathbb{R}^{784 \times 512}$, $\mathbf{b}_1 \in \mathbb{R}^{512}$, $\mathbf{W}_2 \in \mathbb{R}^{512 \times 10}$, $\mathbf{b}_2 \in \mathbb{R}^{10}$,

where $\mathbf{x} \in [0, 1]^{784}$ is the flattened and normalized input image, and σ is the elementwise $\text{ReLU}(x) = \max(x, 0)$ function [Nair and Hinton, 2010].

It follows from equation (6.1) that the target model contains

$$|\theta_{\text{target}}| = \underbrace{784 \cdot 512}_{\mathbf{W}_1} + \underbrace{512}_{\mathbf{b}_1} + \underbrace{512 \cdot 10}_{\mathbf{W}_2} + \underbrace{10}_{\mathbf{b}_2} = 407\,050 \quad (6.2)$$

trainable parameters.

This model is trained for 30 epochs with the Adam optimizer [Kingma and Ba, 2014], reaching an accuracy of 95.28% on the MNIST test set.

Our goal is now to find a rank- r decoupling of $\mathbf{f}$ such that $r \ll 512$ which will result in a smaller network $\hat{\mathbf{f}}$. To this end we will use our CMTF-PSD algorithm (introduced in Chapter 5). The key challenge is that the smaller $\hat{\mathbf{f}}$ should retain the performance of the target $\mathbf{f}$, that is near 95% of accuracy on unseen data.

We run our CMTF-PSD decoupling algorithm with the following hyperparameters:

$$\{\text{rank}=32, \text{niters}=10, \text{df}=23, \text{N}=128\},$$

however, instead of sampling inputs $\mathbf{x}$ at random, we use 128 samples (the size of one training mini-batch) from the training set, since those are representative of the data on which we will use our smaller network.

Using actual data samples to compute our decoupling means that we can compare the decoupled model to the target, and see how often they output the same predictions on these data points. We call this metric *decoupling accuracy*.

To account for poor random initializations, we run the decoupling algorithm 5 times with different random seeds and select the one with the lowest error.

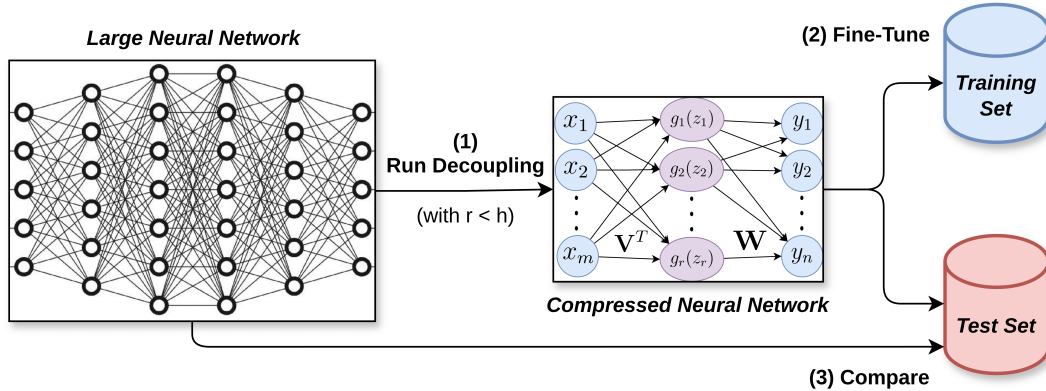


FIGURE 6.2: Neural network compression with tensor decoupling.

After computing $\hat{\mathbf{f}}(\mathbf{x}) = \mathbf{W}\mathbf{g}(\mathbf{V}^T\mathbf{x})$, we freeze the internals $\mathbf{g}$, effectively turning them into fixed but non-identical activation functions. As mentioned earlier, we could also decide to freeze the weights $\mathbf{W}$ and $\mathbf{V}$ instead, which would result in a model that is closer to a Kolmogorov-Arnold network in spirit, but with non-unit weights.

Finally, we *fine-tune*¹ $\mathbf{W}$ and $\mathbf{V}$ for one epoch on the training set, before evaluating our compressed model on the unseen test set. The full methodology is illustrated in Figure 6.2.

¹This term is typically used to denote the latest part of a neural network training pipeline, where the learning rate is reduced. It can also mean adapting the weights of a pretrained model to a specific task.

In general, the compressed network will have $r \cdot (784 + 10)$ trainable parameters, which results in $|\theta_{\text{compressed}}| = 25\,408$ weights for the $r = 32$ case. We therefore have that the compression ratio is approximately 16. Counting the coefficients of the spline internals adds $r \cdot df = 736$ additional parameters.

6.3 Results

Evaluating our decoupled $\hat{\mathbf{f}}$, we find that its accuracy on unseen data is 92.37%, and that it outputs the same predictions as the larger target $\mathbf{f}$ on the batch used for its computation (a perfect decoupling accuracy). The shapes of the 32 internals are shown in Figure 6.3, where we can see that they look different and unlike standard activation functions.

After fine-tuning, we reach an accuracy of 93.86% on the MNIST test set. We have thus derived a model that is more than 15 times smaller compared to the original, while losing less than 2% of its accuracy.

TABLE 6.1: Neural network compression results for different decoupling ranks.

Rank	8	16	32	64
Decoupling Error	0.2658	0.1607	0.1184	0.0974
Decoupling Accuracy (in %)	87.50	99.22	100.00	100.00
Accuracy (in %) <i>Before Fine-Tuning</i>	76.56	89.04	92.37	92.74
Accuracy (in %) <i>After Fine-Tuning</i>	89.40	91.67	93.86	94.17
Compression Ratio	64.08	32.04	16.02	8.01

Results for other ranks are shown in Table 6.1:

- **Decoupling Error** denotes the tensor decomposition error [equation (2.5)].
- **Decoupling Accuracy** measures how often our decoupling predicts the same class as the target, on the 128 samples used for its computation.
- **Accuracy** is the core metric, measuring the accuracy of the compressed model on the MNIST test set (unseen data during either decoupling or fine-tuning).
- **Compression Ratio** is computed as the size (#parameters) of the target network divided by the size of the compressed one.

Note that on this task, it is possible to reach a similar accuracy with a network with the same number of parameters as $\hat{\mathbf{f}}$ trained from scratch. However, in the same setting, this takes hundreds of epochs. Here, we obtained this result starting from a larger network, followed by a decoupling with only 10 iterations, and a single epoch of fine-tuning. Of course, this approach assumes the existence of a target network in the first place.

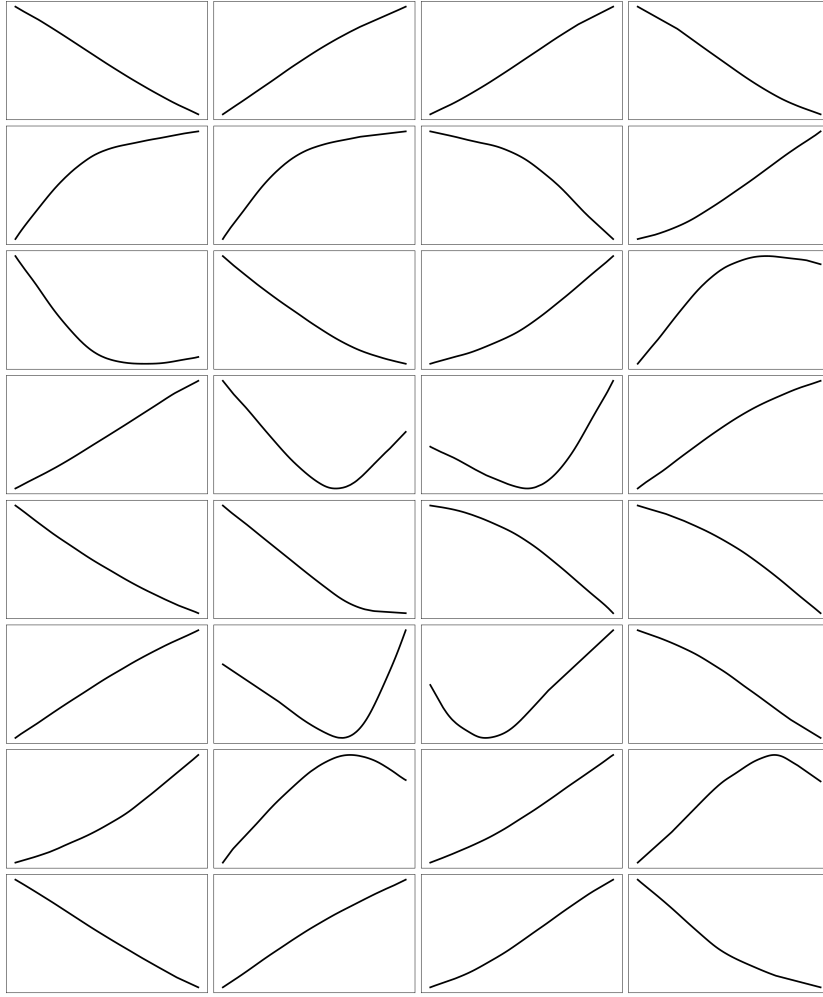


FIGURE 6.3: Internals (activation functions) learned by the rank-32 decoupling.

6.4 Conclusion

In this short chapter, we presented an application of tensor decoupling methods to a real-world problem: the compression of large neural networks. The simple experiment and results presented here show that we can effectively compress large models down to orders-of-magnitude smaller instances, while retaining most of their original performance. These results also demonstrate (or further support) the power of using flexible activation functions in neural networks [Liu et al., 2025].

This kind of approaches could have a substantial impact for low-resource devices and embedded systems, where compute is limited and models must be highly efficient [Menghani, 2023].

The code for reproducing this experiment can be found at
<https://github.com/mrochk/untangle/tree/main/examples/nn>.

Chapter 7

Conclusion

We begin this final chapter by highlighting notable limitations of this work, as well as interesting future research directions in the field of tensor decoupling and its applications to deep learning. Finally, we summarize and conclude this thesis.

7.1 Limitations & Future Work

Lack of comparison to other methods.

The lack of well established benchmarks in the tensor decoupling community makes it challenging to meaningfully compare different methods. Choosing specific synthetic examples can not show that a method is able to perform well on all sorts of multivariate functions. In this work, we also did not compare our algorithm to other non-parametric approaches [Decuyper et al., 2021], or to other tensor decoupling algorithms applied to neural network compression [Zniyed et al., 2021]. A principled way to compare and measure the tradeoffs of decoupling algorithms would make the field of tensor decoupling less opaque and easier to navigate.

Finding the optimal λ^* is computationally expensive.

In Chapters 3 and 5, we use a generalized cross-validation approach [Golub et al., 1979] for finding the optimal P-spline smoothing term λ . However, computing this GCV is expensive, whether we use a grid-search, or an optimization algorithm. In practice, we found that it was sometimes more interesting to simply use a B-spline projection, because being able to try many hyperparameter values and random initializations quickly was more important than the smoothing and non-parametric benefits of penalized splines.

Future work may look into finding an efficient implementation of Minimize in Algorithm 9 by exploring various tradeoffs between speed and performance, updating λ only every $k > 1$ iterations, for example. Otherwise, the faster CMTF-BSD could be seen as a way to quickly find good hyperparameter values, before running the more expensive CMTF-PSD procedure. This would be similar in spirit to how the performance of small machine

learning models is extrapolated to tune the hyperparameters of a larger, more expensive model.

Setting $df \leftarrow N$ is not always optimal in practice.

Although having the benefit of being non-parametric, which in theory, yields the most expressive power, setting $df = N$ in CMTF-PSD makes the algorithm significantly slower. In fact, we found that using $df = 2\sqrt{N}$ by default yielded most benefits for a much lower time per iteration, especially for large N values. Finding good default values and heuristics for df as a function of N is an interesting research direction for future work.

MNIST is an easy machine learning task.

In Chapter 4, we used the MNIST dataset [LeCun et al., 2002a], which is a widely used database for evaluating machine learning models. However, nowadays MNIST can be seen as a "toy example", since even very small and simple models are able to reach a good performance on this task. This example is therefore not representative of real-world machine learning problems. Applying and evaluating tensor decoupling approaches on harder tasks is the next logical step.

Neural network compression only tried on a 2-layer MLP.

In the neural network compression example, we applied the decoupling only to a one hidden layer network, while modern *deep* learning models are made of tens, or hundreds of layers [Srivastava et al., 2015, He et al., 2016]. In this case, the decoupling must be applied to different subsets of the target network. This has been only marginally explored in the literature [Zniyed et al., 2021, De Jonghe et al., 2023, 2026], and not with *very deep* networks. It is not as trivial as simply mapping an entire model to another, as we did in our example.

Finally, the decoupling has only been applied to a basic multilayer perceptron (feedforward) architecture, while modern deep learning models use different types of layers, such as convolutions [LeCun et al., 2002a, He et al., 2016] or attention-based layers [Vaswani et al., 2017]. Applying these methods to convolutional networks and transformer layers has been explored by Zniyed et al. [2021] and De Jonghe et al. [2026]. More applications of tensor decoupling to neural networks, as well as better tooling for deep learning practitioners, will likely follow in the coming years.

Connection with Kolmogorov-Arnold networks.

In Chapter 6 we highlighted the connection between the decoupled representation, multilayer perceptrons, and Kolmogorov-Arnold networks [Liu et al., 2025]. In fact, it could be interesting to dig deeper into this connection by considering tensor decoupling algorithms applied to multilayer perceptrons as mappings of $\text{MLP} \rightarrow \text{KAN}$. This could provide further theoretical insights in the domain of neural network architectures and their tradeoffs.

Fast internals inference.

As discussed in Chapter 6, one advantage of modern neural network architectures is that they can be easily parallelized in SIMD fashion, since they use identical activation functions. The decoupled representation uses flexible activations, meaning that computing them in parallel is not trivial. The tradeoff, of course, is that the decoupled representation, using the power of flexible activations, is able to reach a similar performance with fewer parameters.

Nevertheless, finding ways to represent the decoupled internals in a more efficient manner, and ideally one that allows for parallel computation, would make these approaches even more attractive for practitioners, effectively combining speed with parameter efficiency.

7.2 Conclusion

This thesis has introduced two improvements to the field of tensor decoupling—scaling and smoothing—designed to make these methods more robust, expressive, and less dependent on their hyperparameters.

By implementing a direct rescaling preprocessing step on the input Jacobian tensor, we successfully balance the gradient magnitudes across all outputs. This enables out-of-the-box support for multivariate functions with highly mismatched output ranges, without losing optimization stability or performance.

Concurrently, integrating penalized splines alongside generalized cross-validation effectively removes the computational cost of manual hyperparameter tuning for internal functions’ degrees of freedom. This combination allows the model to maximize its expressivity up to the sample limit, while effectively controlling for overfitting.

Combined, these advancements form the CMTF-PSD algorithm, driving the field of tensor decoupling closer to fully non-parametric approaches.

Finally, the practical viability of these contributions was demonstrated through a neural network compression task, reducing a model’s parameter footprint by more than an order of magnitude while retaining almost all of its original performance.

Bibliography

- Grey Ballard and Tamara G. Kolda. Tensor Decompositions for Data Science. Cambridge University Press, June 2025. URL <https://tensortextbook.com/>.
- Léon Bottou. Large-scale machine learning with stochastic gradient descent. In Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics Paris France, August 22-27, 2010 Keynote, Invited and Contributed Papers, pages 177–186. Springer, 2010.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Richard P Brent. Algorithms for minimization without derivatives. Courier Corporation, 2013.
- J Douglas Carroll and Jih-Jie Chang. Analysis of individual differences in multidimensional scaling via an n-way generalization of “eckart-young” decomposition. Psychometrika, 35(3):283–319, 1970.
- Andrzej Cichocki, Danilo Mandic, Lieven De Lathauwer, Guoxu Zhou, Qibin Zhao, Cesar Caiafa, and Huy Anh Phan. Tensor decompositions for signal processing applications: From two-way to multiway component analysis. IEEE signal processing magazine, 32(2):145–163, 2015.
- Maurice G Cox. The numerical evaluation of b-splines. IMA Journal of Applied mathematics, 10(2):134–149, 1972.
- Peter Craven and Grace Wahba. Smoothing noisy data with spline functions: estimating the correct degree of smoothing by the method of generalized cross-validation. Numerische mathematik, 31(4):377–403, 1978.
- Carl De Boor. On calculating with b-splines. Journal of Approximation theory, 6(1):50–62, 1972.
- Joppe De Jonghe and Mariya Ishteva. Non-parametric b-spline decoupling of multivariate functions. In 2025 33rd European Signal Processing Conference (EUSIPCO), pages 2117–

2121. IEEE, 2025.
- Joppe De Jonghe, Konstantin Usevich, Philippe Dreesen, and Mariya Ishteva. Compressing neural networks with two-layer decoupling. In 9th IEEE International Workshop on Computational Advances in Multi-Sensor Adaptive Processing, pages 226–230. IEEE, 2023.
- Joppe De Jonghe, Van Tien Pham, and Mariya Ishteva. Robust basis spline decoupling for the compression of transformer models. arXiv preprint arXiv:2605.18794, 2026.
- Vin De Silva and Lek-Heng Lim. Tensor rank and the ill-posedness of the best low-rank approximation problem. SIAM Journal on Matrix Analysis and Applications, 30(3):1084–1127, 2008.
- Jan Decuyper, Koen Tiels, Siep Weiland, and Johan Schoukens. Decoupling multivariate functions using a non-parametric filtered cpd approach. IFAC-PapersOnLine, 54(7):451–456, 2021.
- L Peter Deutsch and Allan M Schiffman. Efficient implementation of the smalltalk-80 system. In Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languages, pages 297–302, 1984.
- Philippe Dreesen, Mariya Ishteva, and Johan Schoukens. Decoupling multivariate polynomials using first-order information and tensor decompositions. SIAM Journal on Matrix analysis and Applications, 36(2):864–879, 2015.
- Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. Psychometrika, 1(3):211–218, 1936.
- Paul HC Eilers and Brian D Marx. Flexible smoothing with b-splines and penalties. Statistical science, 11(2):89–121, 1996.
- Walter Gautschi and Gabriele Inglese. Lower bounds for the condition number of vandermonde matrices. Numerische Mathematik, 52(3):241–250, 1987.
- Gene H Golub, Michael Heath, and Grace Wahba. Generalized cross-validation as a method for choosing a good ridge parameter. Technometrics, 21(2):215–223, 1979.
- Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. nature, 585(7825):357–362, 2020.
- Richard A Harshman et al. Foundations of the parafac procedure: Models and conditions for an “explanatory” multi-modal factor analysis. UCLA working papers in phonetics, 16(1):84, 1970.
- Johan Håstad. Tensor rank is np-complete. In International colloquium on automata, languages, and programming, pages 451–460. Springer, 1989.

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.
- Christopher J Hillar and Lek-Heng Lim. Most tensor problems are np-hard. Journal of the ACM (JACM), 60(6):1–39, 2013.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.
- Gabriel Hollander. Multivariate polynomial decoupling in nonlinear system identification. PhD thesis, Vrije Universiteit Brussel, 2017.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.
- Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. SIAM review, 51(3):455–500, 2009.
- Jean Kossaifi, Yannis Panagakis, Anima Anandkumar, and Maja Pantic. Tensorly: Tensor learning in python. Journal of Machine Learning Research, 20(26):1–6, 2019.
- Joseph B Kruskal. Three-way arrays: rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. Linear algebra and its applications, 18(2):95–138, 1977.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 2002a.
- Yann LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In Neural networks: Tricks of the trade, pages 9–50. Springer, 2002b.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. nature, 521(7553):436–444, 2015.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljacic, Thomas Hou, and Max Tegmark. Kan: Kolmogorov–arnold networks. In International conference on learning representations, volume 2025, pages 70367–70413, 2025.
- Gaurav Menghani. Efficient deep learning: A survey on making deep learning models smaller, faster, and better. ACM Computing Surveys, 55(12):1–37, 2023.
- Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In Proceedings of the 27th international conference on machine learning (ICML-10), pages 807–814, 2010.
- Jorge Nocedal and Stephen J Wright. Numerical optimization. Springer, 2006.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imper-

- ative style, high-performance deep learning library. Advances in neural information processing systems, 32, 2019.
- Les Piegl and Wayne Tiller. The NURBS book. Springer Science & Business Media, 2012.
- David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. nature, 323(6088):533–536, 1986.
- Carl Runge et al. Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten. Zeitschrift für Mathematik und Physik, 46(224-243):20, 1901.
- Rupesh K Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. Advances in neural information processing systems, 28, 2015.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- Grace Wahba. Spline models for observational data. SIAM, 1990.
- Yassine Zniyed, Konstantin Usevich, Sebastian Miron, and David Brie. A tensor-based approach for training flexible neural networks. In 2021 55th Asilomar Conference on Signals, Systems, and Computers, pages 1673–1677. IEEE, 2021.

Appendices

Appendix A

The Untangle Python Package



FIGURE A.1: The logo of the Untangle package.

The Untangle library was developed as part of this thesis and served as the primary tool for performing our experiments. We now discuss some implementation details and why we believe this work to be a meaningful contribution to the field.

Untangle is available on GitHub at <https://github.com/mrochk/untangle>.

A.1 Motivation

Given that tensor decoupling is a relatively niche field compared to the broader field of tensor decompositions, there exist many libraries for computing the latter (such as TensorLy [Kossaifi et al., 2019]), but none of them implement tensor decoupling methods. Since no standard implementation of these algorithms exist, researchers and practitioners end up rewriting the same bits of code over and over again, such as collecting the Jacobian, solving the alternating least-squares, fitting the internals... Furthermore, literature in this area traditionally wrote implementations in MATLAB, which is a proprietary programming language, unlike Python.

Nowadays, widely used Python libraries such as Numpy [Harris et al., 2020], Jax [Bradbury et al., 2018] or PyTorch [Paszke et al., 2019] allow working with matrices and tensors very easily, while providing efficient vectorization of linear algebra operations and out-of-the-box support for parallel processors such as GPUs. These can easily serve as a base for building a fast tensor decoupling library.

We believe that building a library that provides tensor decoupling algorithms and utilities may help to popularize these methods in the community, especially in the era of deep learning and given the potential impact that these methods could have in this area (Chapter 6). Untangle implements most of the algorithms discussed in this thesis (including the one introduced in Chapter 5), as well as scaling procedures for preprocessing and common utility functions. It is released under MIT license.

A.2 Implementation Details

Untangle is built on top of Jax [Bradbury et al., 2018], which is itself built on top of Numpy [Harris et al., 2020]. Additionally to basic linear algebra routines provided by the latter, Jax adds support for automatic differentiation (`jax.grad`), just-in-time compilation [Deutsch and Schiffman, 1984] for massive computational speedups (`jax.jit`), automatic function vectorization (`jax.vmap`), and support for hardware accelerators (GPUs).

Typing and shape checking at runtime.

A common issue when working with matrices and tensors is ensuring that the shapes of our objects always match what we expect. When these errors are not caught immediately, this can lead to silent errors or simply bugs that are hard to find.

```
@jaxtyped(typechecker=beartype) # enforce type and shape checking at runtime
def cpd(
    tensor: Float[Array, 'n m N'], # target tensor
    rank: int,                      # cpd rank
    maxiters: int = 100,            # max number of iterations
    key: Optional[Array] = None,    # random seed
    tol: float = 1e-8,              # tolerance for early stopping
) → Tuple[
    Tuple[
        Float[Array, 'n r'], # factor A
        Float[Array, 'm r'], # factor B
        Float[Array, 'N r'], # factors C
    ],
    Float[Array, 'r'],        # weights
    Array,                    # errors
]:
    ...
```

FIGURE A.2: Enforcing tensor shapes at runtime in the signature of our CP-ALS function.

Using the `beartype`¹ runtime type-checker, coupled with `jaxtyping`² allows us to explicitly write in the signature of our functions the expected shapes of our inputs, outputs, as well

¹<https://github.com/beartype/beartype>

²<https://github.com/patrick-kidger/jaxtyping>

as their type. This enforces that the shapes of the tensors are correct and immediately throw an error otherwise, thus allowing the user to catch and fix bugs immediately.

An example of how this looks in practice is shown in Figure A.2, in the signature of our CP-ALS decomposition. In this example, we enforce that the input tensor is real-valued and has the dimensions $n \times m \times N$, but also that all output factors have the correct $\{m, n, N\} \times r$ shape. Furthermore, the symbols m, n, N of the input are automatically matched with the ones in the outputs.

More generally, this approach also makes the source code more readable, and removes the need to specify the expected shapes in the function documentation (docstring) since they can be inferred from the signature of the function directly.

Just-in-time compilation.

One main advantage of using Jax as our core library for tensor operations is that it enables for just-in-time (JIT) compilation, which often results in massive speedups, especially for iterative algorithms that perform the same operations over and over again (such as tensor decoupling procedures). This is particularly easy to implement in our case, since it only requires decorating functions with `@jax.jit`.

```
@jax.jit
@jaxtyped(typechecker=beartype)
def khatri_rao(A: Float[Array, 'm k'], B: Float[Array, 'n k']) -> Float[Array, 'mn k']:
    m, k = A.shape
    n, _ = B.shape
    return (A[:, None, :] * B[None, :, :]).reshape(m*n, k)
```

FIGURE A.3: An example of function annotated with the `@jax.jit` decorator.

An example of this is shown in Figure A.3 with the Khatri-Rao product. The fact that this function is annotated means that the first time this function is called in our program, it gets compiled with XLA³ and cached, such that subsequent calls will be much faster.

A.3 Usage

Figure A.4 shows an example of using the library to compute the rank-3 decoupling of a simple polynomial function with 2 inputs and outputs, using the CMTF-BSD algorithm.

- (1) We define our target function $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, and collect $N = 20$ inputs $\mathbf{X}$, outputs $\mathbf{Y}$ and Jacobians $\mathcal{J}$ using the `collect_information` function.
- (2) We preprocess the Jacobian tensor (Chapter 5) using the `JacobianScaler` class, and obtain our rescaled $\hat{\mathcal{J}}$ and $\hat{\mathbf{Y}}$.
- (3) We can now run our `cmtf_bsd` algorithm, which returns a callable Decoupling object, from which factors and internals can be extracted.

³<https://openxla.org/xla>

- (4) Using our scaler again, we unscale the result such that its outputs can be compared with the target function.
- (5) Finally, we use the `function_error` metric to compute the per-output error in % with respect to the target.

```
import jax.numpy as jnp
from untangle.algorithm import cmtf_bsd
from untangle.scaler import JacobianScaler
from untangle.utils import collect_information, function_error

def target(x):
    a, b = x
    return jnp.array([2*a**3 + b**2 + 1, 3*b**2 + 2*a - 1])

X, Y, J = collect_information(target, N=20)

scaler = JacobianScaler(J, Y)
J_scaled, Y_scaled = scaler.scale()

decoupling, errors = cmtf_bsd(X, Y_scaled, J_scaled, rank=3)
decoupling_unscaled = scaler.unscale(decoupling)

print(f'Errors: {function_error(target, decoupling_unscaled, X)}')
```

FIGURE A.4: Example of using the Untangle library to decouple a cubic polynomial.

More examples can be found at
<https://github.com/mrochk/untangle/tree/main/examples>.

A.4 Future Work

As mentioned in Chapter 7, being able speed up the generalized cross-validation tuning would make CMTF-PSD a much more viable option in practice. Currently, CMTF-BSD is much faster, but taking advantage of Jax’s capabilities could allow us to greatly speed-up the λ tuning, maybe by trading the quality of λ for speed, or using a faster optimizer.

We also mentioned in this chapter that Jax provides support for hardware accelerators such as GPUs, which could lead to massive improvements in performance. In fact, tensor decoupling algorithms perform the same limited number of operations (mainly solving least-squares problems and performing matrix-vector multiplications) many times, which we should be able to parallelize efficiently. We have not yet tried using Untangle on GPUs.

More generally, making these algorithms more efficient would allow us to leverage more data points by increasing N , which would likely lead our algorithms towards better representations.

These ideas leave a lot of room for improving this library in the future, with the goal of popularizing tensor decoupling by making it both more efficient and easier to use.

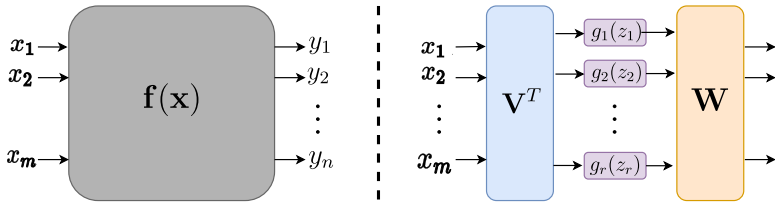
Appendix B

Poster

Presented during the KU Leuven poster event (06/05/2026).

Improving Tensor Decoupling with Normalization & Smoothing

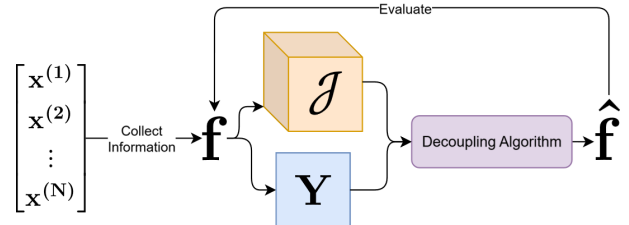
Tensor Decoupling



We collect information about the function (outputs and Jacobians), and use a *decoupling algorithm*, based on the *tensor CP decomposition*.

Tensor decoupling algorithms aim to find a *decoupled representation* of a black-box multivariate function.

$$f(x) \approx Wg(V^T x)$$



- Two Problems:**
- (1) Current methods struggle with functions that output values in different ranges.
 - (2) If certain parameters are chosen incorrectly, this can easily lead to over or under-fitting.

(1) Normalization

$$f(x) = \begin{bmatrix} \cos(4\pi x_1 + x_2) + x_1^2 \\ -\sin(3\pi x_2) + 3x_2^2 \end{bmatrix} \quad \text{error}(y_1) = 3.1\%, \text{error}(y_2) = 2.1\%$$

$$f(x) = \begin{bmatrix} 100 \cos(4\pi x_1 + x_2) + 100x_1^2 \\ -\sin(3\pi x_2) + 3x_2^2 \end{bmatrix} \quad \text{error}(y_1) = 2\%, \text{error}(y_2) = 100\%$$

Function Scaling

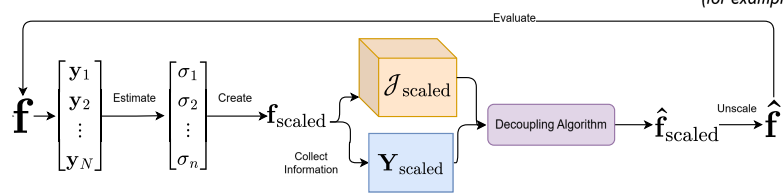
Create a new rescaled function, operate on that one, then unscale the decoupled result to recover the initial function.

$$f_{\text{scaled}}(x) = f(x) \cdot \text{diag}([1/\sigma_1, 1/\sigma_2, \dots, 1/\sigma_n])$$

$$J_{f_{\text{scaled}}}(x) = J_f(x) \cdot \text{diag}([1/\sigma_1, 1/\sigma_2, \dots, 1/\sigma_n])$$

where $\sigma_i = \max(y_i) - \min(y_i)$, or $\text{std}(y_i)$

(for example)



Tensor Normalization

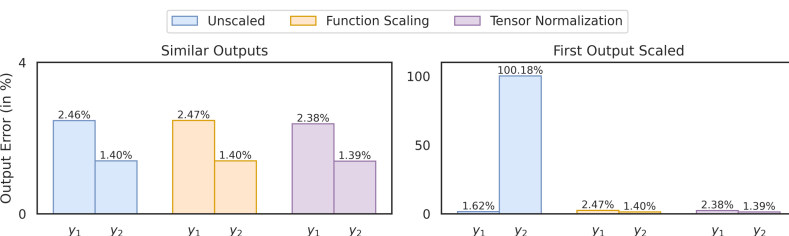
The real equality we are looking to satisfy is the following:

$$\|J_{1,:}\|_F = \|J_{2,:}\|_F = \dots = \|J_{n,:}\|_F$$

Which can be easily obtained via the following normalization:

$$\hat{J}_{i,:} = J_{i,:} \cdot \frac{\sqrt{mN}}{\|J_{i,:}\|_F} \quad \hat{Y}_i = Y_i \cdot \frac{\sqrt{mN}}{\|J_{i,:}\|_F}$$

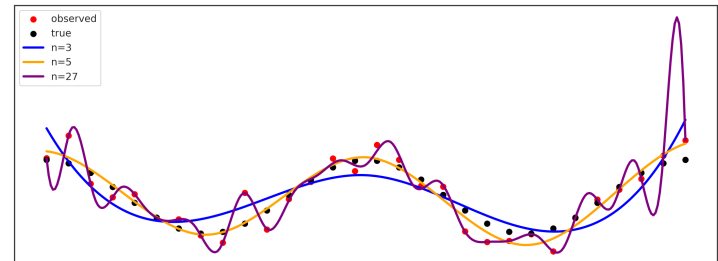
Results



Normalizing the Jacobian tensor improves all decoupling algorithms and is crucial for functions with very different output magnitudes.

(2) Smoothing

The degrees of freedom of the internal B-splines directly impacts the complexity of the functions we can fit. But choosing too many control points leads to overfitting and poor performance.

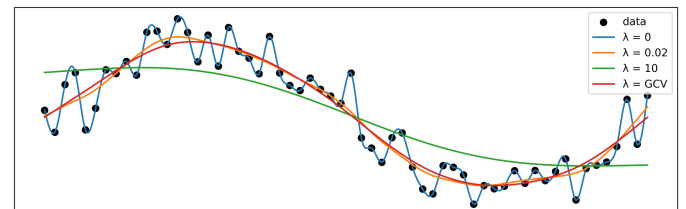


Penalized Splines

P-splines add an additional *smoothing penalty* to the least-squares B-splines objective, this penalty is controlled by the parameter λ .

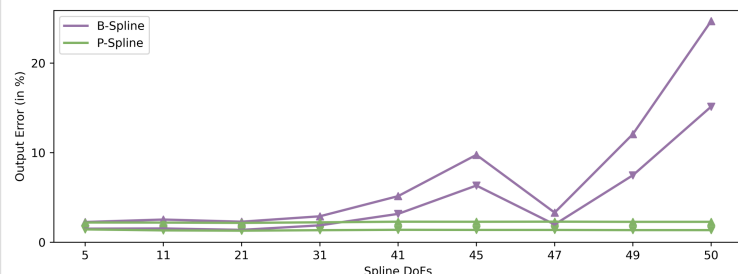
$$\min_c \|y - Bc\|^2 + \lambda \|Dc\|^2$$

Furthermore, λ can be tuned via generalized cross-validation (GCV).



Allowing us to arbitrarily and efficiently increase the expressiveness of our internals, thus effectively making the method *non-parametric*.

Results



P-splines allow to set the DoFs up to the number of sampling points.

Source code available at github.com/mrochku/untangle

Code of conduct and transparency statement on the use of GenAI for KU Leuven-students (academic year 2025–2026)

Generative AI (GenAI) assistance tools can be used to generate various types of content, including text, images, code, video, music, or combinations thereof. Common examples of such tools include ChatGPT, Google Gemini, Microsoft Copilot, Midjourney, Claude.ai, Perplexity.ai, and DALL-E, among others.

This code of conduct is a tool that helps students to be transparent about the use of GenAI and fits within the university's principles on academic integrity.

Important guidelines and remarks

- **Sensitive or personal data:** Some GenAI tools protect your input and sensitive or personal data better than others. There is often no transparency on what the owners of the AI applications do with the data entered. Therefore, **do not enter sensitive or personal data in free GenAI tools**. More info about what sensitive and personal data are, is described in the three confidentiality levels of the KU Leuven data classification model (non-confidential, confidential, strictly confidential). For confidential data you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**. **Try to avoid entering these data. Do so only if strictly necessary, and only to the extent required. For personal data, work with anonymous or pseudonymized data.** Additionally, in case of strictly confidential data this should first be discussed with the teaching staff of the course or your thesis supervisor.
- **Copyrighted materials:** For **lawfully obtained copyrighted material** you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**.

- GenAI assistance may not be used for data or topics covered by a Non-Disclosure Agreement (NDA). Even in the absence of an NDA, certain information may still need to be treated as confidential—for example, due to regulatory requirements or the risk of significant harm to the university if disclosed. In case of doubt, check with your teaching staff or supervisor.
- If your master’s thesis is under embargo, you should first discuss with your supervisor whether the use of GenAI is permitted.
- Before using a GenAI tool, always consider whether its use is responsible, including from a sustainability perspective (e.g. when using GenAI as a search engine, language assistant,...).
- **Take a scientific and critical attitude** when interacting with GenAI assistance and interpreting its output, that may not always be correct.
- As a student you are responsible for complying with Article 84 of the Regulations on Education and Examinations: your report or thesis should reflect your own knowledge, understanding and skills. Be aware that plagiarism rules also apply to (work that is the result of) the use of GenAI assistance tools.

Exam Regulations Article 84: *“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e. copying the work (ideas, texts, structures, designs, images, plans, codes , ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources. Every possession of prohibited resources during an examination (see article 65) is considered an irregularity.”*

- In order to maintain academic integrity and avoid plagiarism, **more information about being transparent on the use of GenAI assistance and about correctly citing and referencing GenAI** can be found on this website for students (Dutch/English).
- **Additional reading: KU Leuven guidelines on responsible use of Generative AI tools, and other information** (Dutch/English)

A few final words

If you are uncertain whether or not you should declare your use of GenAI tools, we suggest that you discuss this with your instructor or supervisor. It is always safer to declare GenAI use, even when it is not strictly required. However, declaring GenAI use does not entail that its use is allowed; the right column in the table below provides more detailed instructions in this regard (code of conduct).

Moreover, advanced AI tools are evolving rapidly, and their capabilities have expanded significantly in a short period of time. As a result, we do not yet have all the answers about their responsible use. Finally, it is important to follow-up on the most recent evolutions in

AI technologies, to have an open mind but also to be a bit cautious, to communicate with instructors, teaching assistants, supervisors and peers, to be as transparent as we can, and to learn together as we move along.

Student name: Maxime Rochkoulets

Student number: R1032552

Please indicate with "X" whether it relates to a course assignment or to the Bachelor's or Master's thesis:

☐ This form is related to a **course assignment**.

Course name: **Course number:**

This form is related to my ☐ **Bachelor's** or ☒ **Master's thesis**.

Title Bachelor's or Master's thesis: Improving Tensor Decoupling
with Scaling and Smoothing

Supervisor: Mariya Ishteva, Hendrick Blockeel

Daily supervisor: Joppe De Jonghe

Please indicate with "X":

☒ I **did not use** any GenAI assistance tool.

☐ I **did use** GenAI Assistance. In this case **specify which ones** (e.g. ChatGPT, M365 Copilot, ...):

GenAI assistance used as/for:	Name of the GenAI tool(s) used If helpful, also describe in which way you were using GenAI related to what is specified as code of conduct.	Code of conduct: For each of the categories below, always take into account the important guidelines and remarks mentioned above (e.g. copyrighted data, sensitive or personal data,...).
As a language assistant for reviewing or improving texts I wrote myself	<i>add text here</i>	This use is similar to using spelling and grammar check tools. In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: When using GenAI tools on texts you did not write yourself to improve the text, you have to refer to the original source or author, otherwise you are committing plagiarism and thus an irregularity.
As a paraphrasing tool	<i>add text here</i>	This use is allowed except when it is prohibited by the teacher or the program of study. You may paraphrase your own text or texts by an author other than yourself and take inspiration from what a GenAI tool or another tool suggests (unless it is not allowed). In general, you do not have to refer to such kind of GenAI use or other paraphrasing tools in the text. However, be careful: If it entails text by an author other than yourself, you are not allowed to include that paraphrased text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity.

For translation aid to improve texts I wrote myself or to better understand text from others	<i>add text here</i>	This use is allowed except when it is prohibited by the teacher or the program of study. It is similar to using translation tools (Google translate, DeepL, ...). In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: • You are not allowed to include that translated text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity. • Always check the translated text for correctness and meaning.
As a search engine to get information on a topic or to search for existing research on the topic	<i>add text here</i>	This use is similar to e.g. a Google search or checking Wikipedia. If you write your own text based on this information, you do not have to refer to the use of GenAI in the text. You only have to refer to the existing research and references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information. The output may not be entirely correct and/or be limited due to the databases it uses. Moreover, knowledge evolves and may change over time; therefore, the database of the GenAI tool may not be up to date. Therefore, verify the information and do not just copy-paste it as you should understand and critically process everything you are writing.

For literature search	<i>add text here</i>	This use is comparable to e.g. a Google Scholar search. You do not have to refer to such kind of GenAI use; you only have to refer to the literature references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the search output is restricted to the database the GenAI tool is built on. After this initial search, look for scientific sources and conduct your own analysis of the source documents. Interpret, analyse and process the information you obtained; verify it and do not just copy-paste it as you should understand and critically process everything you are writing. • Be aware that some GenAI tools may output no or wrong references. As a student you are responsible for further checking and verifying the absence or correctness of references; do not just copy-paste it.
For generating programming code	<i>add text here</i>	Use of GenAI for coding is allowed except when it is prohibited by the teacher or the program of study. If used for coding, correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.

For generating new (research) ideas	<i>add text here</i>	This use of GenAI is allowed except when it is prohibited by the teacher or the program of study. Correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Further verify in this case whether the idea is novel or not. It is likely that it is related to existing work. If so, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus committing an irregularity).
For generating synthetic data	<i>add text here</i>	Use of GenAI for generating synthetic data is allowed, provided that it is methodologically and ethically justifiable, except when it is prohibited by the teacher or the program of study. Always correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Always carefully evaluate the generated synthetic data for quality and possible bias since the output is highly dependent on the quality of the data on which the models are trained.

For generating blocks of text (other than the allowed use without referencing mentioned above)		According to Article 84 of the Regulations on Education and Examinations your text should allow to correctly and properly assess your own knowledge, understanding and skills. Therefore, inserting blocks of text without quotes and a reference to GenAI assistance in your work is not allowed. Be careful: • If it is really needed to insert a block of text from a GenAI tool, for instance because of the nature of your assignment, mention it as a citation by using quotes and correctly mention the use of GenAI assistance in accordance with the instructions page on being transparent about the use of GenAI. • However, in general, such GenAI use should be kept to an absolute minimum; you should always check the original sources.
For generating visuals, video or audio	<i>add text here</i>	Use of GenAI for generating visuals, audio or video is allowed except when it is prohibited by the teacher or the program of study. Be careful: • If used, refer to GenAI for visuals according to the usual referencing style, following the instructions on the page on referencing GenAI. • If you work with existing visuals, audio or video, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus an irregularity). • Explain the usage in the methods section (if there is one) and optionally attach (or link to) the prompts with the full output (or history).

Other use (specify here; this may also include a combination of types of use mentioned above):	<i>add text here</i>	To make sure other use of GenAI is allowed within the course or thesis, and if so, the conditions that may apply, contact the teaching staff of the course or the supervisor of the thesis before-hand and explain the intended GenAI use. Also inform the programme director. Motivate how you would comply with Article 84 of the exam regulations. Explain the use and the added value of the AI tool you consider to use and how it is in accordance with the assignment or thesis and the KU Leuven guidelines on responsible use of Generative AI tools. Depending on the kind of GenAI use, it may be needed to properly reference it in the text, in accordance with the instructions page on being transparent about the use of GenAI.
--	-----------------------------	---